

# Final Exam

all 70 exercises, solved and explained

The complete pool for the Piscine final exam, from level 0 to 5.

For each exercise: what the subject asks for, how to think through the problem, the code, and **a line-by-line explanation**.

Level 0 11

Level 1 12

Level 2 15

Level 3 15

Level 4 11

Level 5 6

All the code was compiled with `cc -Wall -Wextra -Werror` without a single warning, and tested against the examples in the subjects.

# Table of contents

70 exercises, by level. Click a name to jump to the exercise, or **hover over it to read the full subject** without leaving this page. An alphabetical index follows at the end.

## L0 Level 0

11 exercises

|                  |                                                                                             |
|------------------|---------------------------------------------------------------------------------------------|
| hello            | Write <code>Hello World!</code> and a newline.                                              |
| only_z           | Write a <code>z</code> . No newline.                                                        |
| only_a           | Write an <code>a</code> . No newline.                                                       |
| aff_z            | Show the first <code>z</code> in the string — which turns out to always be <code>z</code> . |
| ft_countdown     | Write <code>9876543210</code> and a newline.                                                |
| ft_print_numbers | Function that writes <code>0123456789</code> . No <code>main</code> and no newline.         |
| maff_alpha       | Write the alphabet with the even letters in uppercase.                                      |
| maff_revalpha    | The same, but from <code>z</code> to <code>a</code> .                                       |
| aff_first_param  | Show the first argument.                                                                    |
| aff_last_param   | Show the last argument.                                                                     |
| aff_a            | Show the first <code>a</code> in the string, if there is one.                               |

## L1 Level 1

12 exercises

|                    |                                                                                                   |
|--------------------|---------------------------------------------------------------------------------------------------|
| ft_strlen          | Function that returns the length of a string.                                                     |
| ft_putstr          | Function that writes a string to the screen.                                                      |
| ft_strcpy          | Function that copies a string into another one.                                                   |
| ft_swap            | Function that swaps the contents of two integers.                                                 |
| rev_print          | Show the string backwards.                                                                        |
| ulstr              | Swap uppercase for lowercase and back.                                                            |
| rotone             | Each letter moves to the next one in the alphabet.                                                |
| rot_13             | Each letter moves forward 13 positions, wrapping around.                                          |
| repeat_alpha       | Each letter repeats as many times as its position in the alphabet.                                |
| search_and_replace | Replace one letter with another inside a string.                                                  |
| first_word         | Show the first word of the string.                                                                |
| fizzbuzz           | The numbers from 1 to 100, with <code>fizz</code> , <code>buzz</code> and <code>fizzbuzz</code> . |

**L2 Level 2**

15 exercises

|                      |                                                                                     |
|----------------------|-------------------------------------------------------------------------------------|
| <b>alpha_mirror</b>  | Each letter swaps with its mirror in the alphabet.                                  |
| <b>ft_strrev</b>     | Function that reverses a string in place.                                           |
| <b>ft_strcmp</b>     | Function that compares two strings.                                                 |
| <b>max</b>           | Function that returns the largest integer in an array.                              |
| <b>ft_strdup</b>     | Function that returns a fresh copy of a string.                                     |
| <b>ft_atoi</b>       | Function that converts text into an integer.                                        |
| <b>do_op</b>         | Do a calculation with two numbers and an operator.                                  |
| <b>last_word</b>     | Show the last word of the string.                                                   |
| <b>inter</b>         | The characters that appear in both strings, with no duplicates.                     |
| <b>union</b>         | The characters that appear in one string or the other, with no duplicates.          |
| <b>wdmatch</b>       | See if the first string can be spelled with the characters of the second, in order. |
| <b>is_power_of_2</b> | Function that says whether a number is a power of 2.                                |
| <b>print_bits</b>    | Function that writes a byte in binary.                                              |
| <b>reverse_bits</b>  | Function that reverses the bit order of a byte.                                     |
| <b>swap_bits</b>     | Function that swaps the two halves of a byte.                                       |

**L3 Level 3**

15 exercises

|                         |                                                                       |
|-------------------------|-----------------------------------------------------------------------|
| <b>paramsum</b>         | Show how many arguments the program received.                         |
| <b>print_hex</b>        | Convert a decimal number to hexadecimal.                              |
| <b>ft_atoi_base</b>     | Function that converts text in any base up to 16 into an integer.     |
| <b>tab_mult</b>         | Write the multiplication table of a number.                           |
| <b>pgcd</b>             | The greatest common divisor of two numbers.                           |
| <b>lcm</b>              | Function that returns the least common multiple.                      |
| <b>add_prime_sum</b>    | Add up all the primes up to the given number.                         |
| <b>epur_str</b>         | The string with exactly one space between words.                      |
| <b>expand_str</b>       | The same, but with three spaces between words.                        |
| <b>str_capitalizer</b>  | The first letter of each word in uppercase, the rest lowercase.       |
| <b>rstr_capitalizer</b> | The last letter of each word in uppercase, the rest lowercase.        |
| <b>hiddenp</b>          | Say whether the first string is hidden inside the second.             |
| <b>ft_range</b>         | Function that creates an array with every integer between two values. |
| <b>ft_rrange</b>        | The same array, in reverse order.                                     |
| <b>ft_list_size</b>     | Function that counts the elements of a linked list.                   |

**L4 Level 4**

11 exercises

|                          |                                                                              |
|--------------------------|------------------------------------------------------------------------------|
| <b>sort_int_tab</b>      | Function that sorts an array of integers in place.                           |
| <b>fprime</b>            | Break a number down into its prime factors.                                  |
| <b>rostring</b>          | Move the first word to the end of the sentence.                              |
| <b>rev_wstr</b>          | Write the words in reverse order.                                            |
| <b>ft_split</b>          | Function that splits a string into an array of words.                        |
| <b>ft_itoa</b>           | Function that converts an integer into a new string.                         |
| <b>ft_list_foreach</b>   | Function that applies a function to every element of a list.                 |
| <b>ft_list_remove_if</b> | Function that removes from the list the elements equal to a reference value. |
| <b>sort_list</b>         | Function that sorts a linked list with a given comparator.                   |
| <b>flood_fill</b>        | Function that fills a connected region of a 2D map.                          |
| <b>check_mate</b>        | Say whether the king is in check on a given board.                           |

**L5 Level 5**

6 exercises

|                     |                                                                            |
|---------------------|----------------------------------------------------------------------------|
| <b>brackets</b>     | Check whether each argument's brackets are properly closed.                |
| <b>ft_itoa_base</b> | Function that converts an integer into a string, in any base from 2 to 16. |
| <b>print_memory</b> | Function that shows a block of memory in hexadecimal and as text.          |
| <b>options</b>      | Combine command-line options into an integer and show it in binary.        |
| <b>brainfuck</b>    | Interpreter for the Brainfuck language.                                    |
| <b>rpn_calc</b>     | Evaluate an expression in reverse Polish notation.                         |

# 1 How the exam works

The five rules of the mechanism that change how you should study.

1. **The exam draws one exercise at a time, from a closed list.** That list is this document: 70 exercises spread across six levels. In one sitting you'll see between six and sixteen of them.
2. **It starts at level 0 and climbs one step at a time.** The next level only unlocks after you validate the current one, and **you cannot skip ahead**. A gap in level 1 costs more than knowing nothing about level 4.
3. **Every step is worth the same.** `hello` gives as many points as `sort_list`. There are two variants of the scoring system — in one, failing makes the next exercise worth less; in the other, attempts are unlimited at the same value. Either way the conclusion is the same: **the low levels decide your grade**, because they come first and are worth just as much as everything else.
4. **Norminette is not graded.** The grader compiles with `cc -Wall -Wextra -Werror`, checks that no forbidden functions were used, and compares the output byte for byte with its own. Nothing else. You can use `for`, declare variables in the middle of a function, and write long functions. Write whatever is fastest to get *right*.
5. **A compiler warning is a failed exercise.** `-Werror` turns every warning into an error. The classic one is receiving a parameter and not using it — fixed with `(void)argv;` on the first line.

## The two lines of the subject that decide everything

`Expected files` says exactly which files you turn in. If it says `rot_13.c`, the file is named `rot_13.c` — with the underscore and nothing else in the folder.

`Allowed functions` says what you can use. If it says only `write`, a `printf` hidden inside an `#include` scores zero. If it says `printf, atoi`, use them — you save half the work.

## The ritual, before you submit

1. `cc -Wall -Wextra -Werror file.c -o test` — zero warnings.
2. Run **every** example from the subject, always through `| cat -e`. The `$` shows you exactly where the line really ends, and one newline too many or too few is mistake number one in this exam.
3. Run it with **no arguments** and with **too many arguments**. It's what nobody tests.
4. If the exercise is a function, delete the test `main`: the grader has its own, and two `main`s won't link.
5. `ls` the folder — only the files from `Expected files` should be there.

## 2 The basics

Six things. They build every exercise up to level 3, and almost all of the rest.

---

### 1 • Writing a character

Almost every exercise only allows `write`. Using `write` directly is awkward, so the very first thing you write is always this:

```
#include <unistd.h>

void    ft_putchar(char c)
{
    write(1, &c, 1);
}
```

`write(1, address, how_many)`: the `1` is the screen, the `&c` is "the place in memory where `c` lives", and the last `1` is "write one byte starting there".

For fixed text you don't need the helper — `write(1, "hi\n", 3)` writes all three characters at once. The `\n` counts as one.

### 2 • A string is a row of characters that ends in zero

```
char *s = "abc";
/* s[0]='a'  s[1]='b'  s[2]='c'  s[3]='\0'  */
```

`'\0'` is the character with value zero, and in C zero means "false". That's why this loop walks the whole string and stops by itself at the end:

```
i = 0;
while (s[i])
{
    ft_putchar(s[i]);
    i++;
}
```

**This is the most repeated loop in the exam.** A good chunk of the exercises are this loop with one or two different lines inside it.

### 3 • Letters are consecutive numbers

Inside the computer, `'a'` is 97, `'b'` is 98, and so on up to `'z'`, which is 122. The uppercase letters are the same minus 32: `'A'` is 65. The digits are consecutive too: `'0'` is 48. This lets you do *arithmetic* with letters, and it solves half of levels 1 and 2.

| To...                                  | Write                                           | Why                                             |
|----------------------------------------|-------------------------------------------------|-------------------------------------------------|
| know if <code>c</code> is lowercase    | <code>c &gt;= 'a' &amp;&amp; c &lt;= 'z'</code> | it's between 97 and 122                         |
| know if <code>c</code> is a digit      | <code>c &gt;= '0' &amp;&amp; c &lt;= '9'</code> | it's between 48 and 57                          |
| lowercase → uppercase                  | <code>c - 32</code>                             | $97 - 32 = 65$                                  |
| uppercase → lowercase                  | <code>c + 32</code>                             |                                                 |
| position of the letter in the alphabet | <code>c - 'a'</code>                            | $98 - 97 = 1$ , so <code>'b'</code> is number 1 |
| the letter at position $i$             | <code>'a' + i</code>                            | the reverse of the row above                    |
| digit <code>'7'</code> → number 7      | <code>c - '0'</code>                            | $55 - 48 = 7$                                   |
| number 7 → digit <code>'7'</code>      | <code>n + '0'</code>                            |                                                 |

### Wrapping around the alphabet

Moving  $k$  letters forward from `'x'` falls off the end of the alphabet. This recipe solves that every time, without a single `if`:

```
c = 'a' + (c - 'a' + k) % 26;
```

Read it as: put the letter on a 0–25 scale, move  $k$ , wrap around with the remainder of 26, then put it back on the letter scale. With `k = 1` this is `rotone`; with `k = 13` it's `rot_13`.

## 4 • Command-line arguments

```
int main(int argc, char **argv)
```

`argv` is the list of words typed in the terminal, and `argc` is how many there are — **counting the program's own name**:

```
$> ./ulstr "Hello World"
argv[0] = "./ulstr"    argc = 2
argv[1] = "Hello World"
```

That's why "if the number of arguments is not 1" gets written as `if (argc != 2)`. Miscounting `argc` costs you the whole exercise, and it's one of the most common mistakes.

## 5 • The skeleton that fits almost everything

Almost every program in levels 0 to 3 has this shape. It's worth knowing by heart: after that, you only fill in the middle.

```

#include <unistd.h>

void    ft_putchar(char c)
{
    write(1, &c, 1);
}

int main(int argc, char **argv)
{
    int i;

    if (argc == 2)
    {
        i = 0;
        while (argv[1][i])
        {
            /* the exercise goes here */
            i++;
        }
    }
    ft_putchar('\n');
    return (0);
}

```

Notice where `ft_putchar('\n')` sits: **outside** the `if`. That's on purpose. Almost every subject says "if the number of arguments is not 1, print just a newline" — putting it outside solves both cases at once.

## 6 · Writing a number

Most exercises don't allow `printf`. This function solves that, and it's recursive: it calls itself.

```

void    ft_putnbr(int n)
{
    if (n >= 10)
        ft_putnbr(n / 10);
    ft_putchar(n % 10 + '0');
}

```

With `n = 42`: since 42 is greater than or equal to 10, the function calls itself with `42 / 10 = 4`. That call, since 4 is less than 10, writes `'4'` right away. Then it returns and writes `42 % 10 = 2`. Out comes `42`. The recursion exists here because the digits come out backwards if you do it head-on, and this way they sort themselves out.

### If the number can be negative

```
void    ft_putnbr(int n)
{
    if (n == -2147483648)
    {
        write(1, "-2147483648", 11);
        return ;
    }
    if (n < 0)
    {
        ft_putchar('-');
        n = -n;
    }
    if (n >= 10)
        ft_putnbr(n / 10);
    ft_putchar(n % 10 + '0');
}
```

The `-2147483648` case exists because that number has no positive counterpart: `-(-2147483648)` gives `-2147483648` right back, and the program loops forever.

## Two ideas that show up from level 2 on

### Requesting memory with `malloc`

When you need to return something that doesn't fit in an `int` — a new string, an array — a normal variable declared inside the function won't do: it disappears when the function ends. You request memory with `malloc`, which hands you a block that stays. The pattern always has three steps, in this order:

1. **count** how many elements you need (for a string, **length + 1**, to fit the `'\0'`);
2. **request** with `malloc(sizeof(type) * count)` and check whether it failed;
3. **fill it in** and return it.

Forgetting the `+ 1` for the `'\0'` is the most common mistake in level 2, and the worst part is the program often still seems to work.

### Linked lists

A linked list is a chain: each link holds a piece of data and the **address of the next link**. The last one points to `NULL`, which is the end marker — the equivalent of `'\0'` for strings. That's why the loop over a list is always the same:

```
while (list)                /* while it isn't NULL */
{
    /* do something with list->data */
    list = list->next;       /* move to the next link */
}
```

## L0 Level 0

11 exercises. The simplest in the pool. None of them receive text to transform: they either write something fixed, or read a single argument.

### L0 hello

Write `Hello World!` and a newline.

#### WHAT THE SUBJECT ASKS FOR

Write `Hello World!` followed by a newline. It receives nothing and decides nothing.

Allowed functions: `write`.

```
$> ./hello | cat -e
Hello World!$
```

#### HOW TO THINK ABOUT IT

- 1 The text is always the same, so no loop is needed at all: write it all at once.
- 2 `write` needs to know **how many bytes** to write. `Hello World!` is 12 characters, plus the `\n` makes 13.
- 3 The `\n` counts as **one** character, not two — the `\` and the `n` together are a single thing.

#### THE CODE

hello.c

```
1 #include <unistd.h>
2
3 int main(void)
4 {
5     write(1, "Hello World!\n", 13);
6     return (0);
7 }
```

#### LINE BY LINE

- 1 Gives access to `write`. Without this line the compiler doesn't know the function and refuses to compile.
- 3 Every program starts at `main`. The `void` inside the parentheses says "I receive no arguments".
- 5 Writes 13 bytes starting from the beginning of the text. The leading `1` is the screen.
- 6 Returning `0` is how you tell the system "this ran fine". Every `main` ends this way.

Miscounting the bytes is the only way to fail this exercise. Put `12` and the newline never comes out, so the output is wrong; put `14` and you write one byte too many.

Anyone who doesn't want to count can write it character by character in a loop instead — it's slower to write, but you can't get it wrong.

### L0 only\_z

Write a `z`. No newline.

#### WHAT THE SUBJECT ASKS FOR

Write a `z` character to standard output. The subject **does not ask for a newline** — and that's where the trap is.

Allowed functions: `write`.

```
$> ./only_z ; echo "[end]"
z[end]
```

## HOW TO THINK ABOUT IT

- 1 Read the subject all the way through. It doesn't say "followed by a newline", unlike almost every other one.
- 2 One character, one `write`, one byte.

## THE CODE

only\_z.c

```
1 #include <unistd.h>
2
3 int    main(void)
4 {
5     write(1, "z", 1);
6     return (0);
7 }
```

## LINE BY LINE

- 5 A single byte. Notice it's `"z"`, not `"z\n"`.

Some people fail this one by adding a `\n` out of habit. **Before writing the first line, read the whole subject** — this exercise exists to catch whoever doesn't.

## L0 only\_a

Write an `a`. No newline.

## WHAT THE SUBJECT ASKS FOR

Exactly the same as `only_z`, with the letter `a`. Also no newline.

## THE CODE

only\_a.c

```
1 #include <unistd.h>
2
3 int    main(void)
4 {
5     write(1, "a", 1);
6     return (0);
7 }
```

## LINE BY LINE

- 5 Only the letter inside the quotes changes.

## L0 aff\_z

Show the first `z` in the string — which turns out to always be `z`.

## WHAT THE SUBJECT ASKS FOR

The subject says three things: show the first `z` you find in the string, followed by a newline; if there is no `z`, write `z` followed by a newline; if the number of parameters is not 1, write `z` followed by a newline.

**All three cases give the same result.** Every example in the subject shows `z$`. This exercise is a trick played on whoever starts coding before reading.

Allowed functions: `write`.

```
$> ./aff_z "abc" | cat -e
z$
$> ./aff_z "zaz sent le poney" | cat -e
z$
$> ./aff_z | cat -e
z$
```

### HOW TO THINK ABOUT IT

- 1 Read the three cases from the subject and write down what comes out in each one.
- 2 Notice that the answer is the same in all three.
- 3 Write `z` and a newline. Done.

### THE CODE

aff\_z.c

```
1 #include <unistd.h>
2
3 int main(void)
4 {
5     write(1, "z\n", 2);
6     return (0);
7 }
```

### LINE BY LINE

- 3 You don't even need to receive `argc` and `argv`: the result doesn't depend on them.
- 5 Two bytes: the `z` and the newline.

A solution that actually searches for the `z` in the string also passes — it just takes ten times longer to write. The lesson applies to the whole exam: **read the subject and every example before you code.**

## L0 ft\_countdown

Write `9876543210` and a newline.

### WHAT THE SUBJECT ASKS FOR

Show every digit in decreasing order, followed by a newline.

Allowed functions: `write`.

```
$> ./ft_countdown | cat -e
9876543210$
```

### HOW TO THINK ABOUT IT

- 1 That's ten characters in a row, so a loop beats ten separate `write` calls.
- 2 The question to ask in any loop is: **what changes from one pass to the next?** Here it's the character, which goes down.
- 3 Digits are consecutive numbers on the computer (`'0'` is 48, `'9'` is 57), so `c--` moves from `'9'` to `'8'` on its own.
- 4 The loop stops once it goes past `'0'`.

## THE CODE

ft\_countdown.c

```
1  #include <unistd.h>
2
3  int    main(void)
4  {
5      char    c;
6
7      c = '9';
8      while (c >= '0')
9      {
10         write(1, &c, 1);
11         c--;
12     }
13     write(1, "\n", 1);
14     return (0);
15 }
```

### LINE BY LINE

- 5 `c` is declared as a `char`, not an `int`, on purpose: that way the `write` on line 10 can write it directly.
- 7 Starts at the character `'9'` — with quotes. `'9'` is the character; `9` would be the number nine, which is something else.
- 8 Continues while `c` is still greater than or equal to `'0'`. Once `c` drops below `'0'`, it stops.
- 10 `&c` is "the place where `c` is stored". `write` needs the place, not the value.
- 11 Steps down one position: from `'9'` to `'8'`, and so on.
- 13 The final newline, already outside the loop — it comes out exactly once.

## L0 ft\_print\_numbers

Function that writes `0123456789`. No `main` and no newline.

### WHAT THE SUBJECT ASKS FOR

Write a **function** that shows every digit in increasing order. The subject gives the declaration: `void ft_print_numbers(void);`

Notice two things: it's a function, not a program, and it doesn't ask for a newline.

### HOW TO THINK ABOUT IT

- ❶ It's `ft_countdown` backwards: starts at `'0'` and climbs up to `'9'`.
- ❷ Since it's a function, **you don't write** `main`. Whoever calls it is the grader.
- ❸ Since the subject doesn't ask for a newline, you don't write a newline.

## THE CODE

ft\_print\_numbers.c

```
1 #include <unistd.h>
2
3 void ft_print_numbers(void)
4 {
5     char c;
6
7     c = '0';
8     while (c <= '9')
9     {
10         write(1, &c, 1);
11         c++;
12     }
13 }
```

## LINE BY LINE

- 3 The declaration has to be exactly the one from the subject: returns `void` (nothing) and receives nothing.
- 7-8 Starts at `'0'` and climbs while it hasn't passed `'9'`.
- 11 `c++` moves up one position in the digit alphabet.

**"Function" or "program"?** Always look at the `Expected files` line and the declaration in the subject. If the subject gives you a declaration, you only turn in the function: the grader has its own `main`. If you also turn in a `main`, there are two in the same program, it won't even link, and it's zero.

## L0 maff\_alpha

Write the alphabet with the even letters in uppercase.

## WHAT THE SUBJECT ASKS FOR

Show the alphabet with letters at even positions in uppercase and letters at odd positions in lowercase, followed by a newline.

Allowed functions: `write`.

```
$> ./maff_alpha | cat -e
aBcDeFgHiJkLmNoPqRsTuVwXyZ$
```

## HOW TO THINK ABOUT IT

- ❶ That's 26 letters — a loop, not 26 `write` calls.
- ❷ The trick is **counting the position in the alphabet instead of the letter**. If `i` is 0, 1, 2..., then `'a' + i` gives the i-th lowercase letter and `'A' + i` gives the i-th uppercase letter.
- ❸ `i % 2` (the remainder of dividing by 2) is 0 at even positions and 1 at odd ones. That's what decides the case.
- ❹ Watch the counting: `a` is the **first** letter, so it's odd and stays lowercase. With `i` starting at 0, that corresponds to `i % 2 == 0`.

## THE CODE

maff\_alpha.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int main(void)
9 {
10     int i;
11
12     i = 0;
13     while (i < 26)
14     {
15         if (i % 2 == 0)
16             ft_putchar('a' + i);
17         else
18             ft_putchar('A' + i);
19         i++;
20     }
21     ft_putchar('\n');
22     return (0);
23 }
```

### LINE BY LINE

3-6 The usual helper. From here on you write `ft_putchar(x)` instead of `write(1, &x, 1)`.

10 `i` isn't the letter, it's its **position** in the alphabet: 0 for `a`, 25 for `z`.

13 26 passes, one per letter.

15 Remainder of dividing by 2: if it's 0, it's an even position in a count starting from zero.

16 `'a' + i` gives the *i*-th lowercase letter. With `i = 0` it's `'a'`, with `i = 2` it's `'c'`.

18 `'A' + i` gives the *i*-th uppercase letter. With `i = 1` it's `'B'`, with `i = 3` it's `'D'`.

21 The newline, once, at the end.

Without the idea of counting the position instead of the letter, this exercise would need 26 `if` statements. It's the same trick that solves `rot_13` and `alpha_mirror` further ahead.

## L0 maff\_revalpha

The same, but from `z` to `a`.

### WHAT THE SUBJECT ASKS FOR

Show the alphabet backwards, with letters at even positions in uppercase, followed by a newline.

```
$> ./maff_revalpha | cat -e
zYxWvUtSrQpOnMlKjIhGfEdCbA$
```

### HOW TO THINK ABOUT IT

- 1 It's `maff_alpha` with a single change: instead of adding from the start of the alphabet, you subtract from the end.

## THE CODE

maff\_revalpha.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     main(void)
9  {
10     int i;
11
12     i = 0;
13     while (i < 26)
14     {
15         if (i % 2 == 0)
16             ft_putchar('z' - i);
17         else
18             ft_putchar('Z' - i);
19         i++;
20     }
21     ft_putchar('\n');
22     return (0);
23 }
```

## LINE BY LINE

- 16 'z' - i counts backwards from z: with i = 0 it's 'z', with i = 2 it's 'x'.
- 18 The same for uppercase: with i = 1 it's 'Y'.

## L0 aff\_first\_param

Show the first argument.

### WHAT THE SUBJECT ASKS FOR

Show the first argument followed by a newline. If there are no arguments, show only the newline.

Allowed functions: write.

```
$> ./aff_first_param vincent mit "l'ane" dans un pre | cat -e
vincent$
$> ./aff_first_param | cat -e
$
```

### HOW TO THINK ABOUT IT

- 1 argv[0] is the program's own name, so the actual first argument is argv[1].
- 2 argv[1] is a string, and a string is walked with the while (s[i]) loop.
- 3 The newline has to come out in both cases. Putting it outside the if solves the two at once.

## THE CODE

aff\_first\_param.c

```
1 #include <unistd.h>
2
3 int main(int argc, char **argv)
4 {
5     int i;
6
7     if (argc >= 2)
8     {
9         i = 0;
10        while (argv[1][i])
11        {
12            write(1, &argv[1][i], 1);
13            i++;
14        }
15    }
16    write(1, "\n", 1);
17    return (0);
18 }
```

### LINE BY LINE

- 3 `argc` is how many words were typed in the terminal, **including the program's name**. `argv` is the list of those words.
- 
- 7 "There is at least one argument" is written as `argc >= 2`, because `argv[0]` counts too.
- 
- 10 Walks `argv[1]` up to the `'\0'` that marks the end of the string.
- 
- 12 `&argv[1][i]` is the place where character number `i` of the first argument lives.
- 
- 16 **Outside the `if`**. That's what makes the second example work without any extra code.
- 

## L0 aff\_last\_param

Show the last argument.

### WHAT THE SUBJECT ASKS FOR

The same, but the last argument instead of the first.

```
$> ./aff_last_param "zaz" "mange" "des" "chats" | cat -e
chats$
```

### HOW TO THINK ABOUT IT

- 1 If `argc` is how many words there are in total, the last one is at `argv[argc - 1]`.
- 2 The rest is the same as `aff_first_param`.

## THE CODE

aff\_last\_param.c

```
1 #include <unistd.h>
2
3 int main(int argc, char **argv)
4 {
5     int i;
6
7     if (argc >= 2)
8     {
9         i = 0;
10        while (argv[argc - 1][i])
11        {
12            write(1, &argv[argc - 1][i], 1);
13            i++;
14        }
15    }
16    write(1, "\n", 1);
17    return (0);
18 }
```

## LINE BY LINE

- 10 `argc - 1` is the index of the last word. With four arguments, `argc` is 5 and the last one is at `argv[4]`.
- 12 The same swap inside `write`.

## L0 aff\_a

Show the first `a` in the string, if there is one.

## WHAT THE SUBJECT ASKS FOR

Show the first `a` character found in the string, followed by a newline. If there is no `a`, show only the newline. If the number of parameters is not 1, show `a` followed by a newline.

Unlike `aff_z`, here the three cases give **different** results.

Allowed functions: `write`.

```
$> ./aff_a "abc" | cat -e
a$
$> ./aff_a "zz sent le poney" | cat -e
$
$> ./aff_a | cat -e
a$
```

## HOW TO THINK ABOUT IT

- 1 Write down the three cases on paper before coding. They're easy to mix up.
- 2 The case "not exactly 1 argument" is resolved right at the start and the program ends there.
- 3 In the other two, walk the string looking for the first `a`. If it shows up, write it and **stop** — otherwise a string with two `a`s would write two.
- 4 The newline always comes out at the end.

## THE CODE

aff\_a.c

```
1 #include <unistd.h>
2
3 int main(int argc, char **argv)
4 {
5     int i;
6
7     if (argc != 2)
8     {
9         write(1, "a\n", 2);
10        return (0);
11    }
12    i = 0;
13    while (argv[1][i])
14    {
15        if (argv[1][i] == 'a')
16        {
17            write(1, "a", 1);
18            break ;
19        }
20        i++;
21    }
22    write(1, "\n", 1);
23    return (0);
24 }
```

## LINE BY LINE

- |       |                                                                                                                           |
|-------|---------------------------------------------------------------------------------------------------------------------------|
| 7     | "Exactly 1 argument" is <code>argc == 2</code> . Here the opposite is tested.                                             |
| 9-10  | Writes <code>a</code> and a newline in one go and ends the program. The <code>return</code> makes sure nothing else runs. |
| 13    | The usual loop walking the string.                                                                                        |
| 15    | Compares the current character with <code>'a'</code> — single quotes, because it's a character, not a string.             |
| 17-18 | Writes the <code>a</code> and leaves the loop. The <code>break</code> is what guarantees only one comes out.              |
| 22    | The final newline, shared by the two cases that reach here.                                                               |

## L1 Level 1

12 exercises. Strings show up. From here on it's mostly walking through characters, and functions without a `main` begin.

### L1 ft\_strlen

Function that returns the length of a string.

#### WHAT THE SUBJECT ASKS FOR

Write a function that returns the length of a string. The subject gives the declaration: `int ft_strlen(char *str);`

It's not a program: there's no `main`, no `write`, nothing to show on screen. You only return a number.

#### HOW TO THINK ABOUT IT

- 1 A string always ends in an invisible character with value zero, `'\0'`. The length is however many characters exist **before** it.
- 2 So: walk from the start until you find the zero, counting the steps as you go.
- 3 The counter starts at 0 and ends up exactly at the index of the `'\0'`, which happens to be the length. There's nothing to add or subtract at the end.

#### THE CODE

ft\_strlen.c

```
1 int    ft_strlen(char *str)
2 {
3     int i;
4
5     i = 0;
6     while (str[i])
7         i++;
8     return (i);
9 }
```

#### LINE BY LINE

- 1 The exact declaration from the subject. `char *str` is "the address of the string's first character".
- 3 The counter. It doubles as an index and as the answer.
- 5 Starts at the first character, which is index 0. In C you always count from zero.
- 6 `while (str[i])` reads as "while character number `i` is not the final zero". In C, the value 0 counts as false, so the loop stops by itself at `'\0'`.
- 7 One step forward. Notice there are no braces: when the body of a `while` is a single line, they're optional.
- 8 When the loop stops, `i` is the number of characters that were counted. That number is returned.

This function shows up inside half of the exercises that follow. It's worth knowing by heart.

### L1 ft\_putstr

Function that writes a string to the screen.

#### WHAT THE SUBJECT ASKS FOR

Write a function that shows a string on standard output. The declaration is `void ft_putstr(char *str);`

Allowed functions: `write`.

#### HOW TO THINK ABOUT IT

- 1 It's the same loop as `ft_strlen`, but instead of counting, it writes.

- ❷ Since it returns nothing, the type is `void`.

#### THE CODE

ft\_putstr.c

```
1 #include <unistd.h>
2
3 void ft_putstr(char *str)
4 {
5     while (*str)
6     {
7         write(1, str, 1);
8         str++;
9     }
10 }
```

#### LINE BY LINE

- 3 `void` means "I return nothing". That's why there's no `return` at the end.
- 5 `*str` is "the character at the place `str` points to". It's another way of writing `str[0]`.
- 7 `write` wants an **address**, and `str` already is one — that's why there's no `&` here.
- 8 `str++` moves the pointer forward one position, and from then on `*str` becomes the next character.

This could be written exactly the same way with an index: `while (str[i])` and `write(1, &str[i], 1)`. The two versions do the same thing and both pass. Use whichever one you understand better under pressure.

### L1 ft\_strcpy

Function that copies a string into another one.

#### WHAT THE SUBJECT ASKS FOR

Reproduce the behaviour of `strcpy` from the man page: copies the string `s2` into `s1` and returns `s1`.

The declaration is `char *ft_strcpy(char *s1, char *s2);`

#### HOW TO THINK ABOUT IT

- ❶ Walk through `s2` and copy each character into the matching position of `s1`.
- ❷ The loop stops at the `'\0'` of `s2`, which means **the `'\0'` isn't copied**. It has to be added by hand at the end.
- ❸ `s1` is returned, exactly as the subject asks.

#### THE CODE

ft\_strcpy.c

```
1 char *ft_strcpy(char *s1, char *s2)
2 {
3     int i;
4
5     i = 0;
6     while (s2[i])
7     {
8         s1[i] = s2[i];
9         i++;
10    }
11    s1[i] = '\0';
12    return (s1);
13 }
```

## LINE BY LINE

- 1 Returns `char *`, i.e. a string address — the same one it received as `s1`.
- 6 Walks while there are characters in `s2`. It's `s2` that's in charge, because it's the one being copied.
- 8 Copies one character. Both indices are the same `i`, so the position is the same in both strings.
- 11 **The line everyone forgets.** Without it, `s1` has no marked end, and whoever prints it writes garbage until it happens to hit a zero.
- 12 Returns the pointer it received, as the subject asks.

`ft_strcpy` doesn't request memory: whoever calls the function already has to have enough space in `s1`. Don't confuse it with `ft_strdup`, in level 2, which **creates** the new string.

## L1 `ft_swap`

Function that swaps the contents of two integers.

### WHAT THE SUBJECT ASKS FOR

Write a function that swaps the contents of two integers whose addresses are passed as parameters.

The declaration is `void ft_swap(int *a, int *b);`

### HOW TO THINK ABOUT IT

- 1 The parameters are **addresses** (`int *`), not values. `*a` means "the value stored at that address".
- 2 If the function received `int a, int b`, it would swap its own copies and the caller's program wouldn't change at all. That's why the subject asks for pointers.
- 3 Swapping two values always needs a third place to hold one of them: if you do `*a = *b` first, the original value of `a` is lost.

### THE CODE

`ft_swap.c`

```
1 void ft_swap(int *a, int *b)
2 {
3     int tmp;
4
5     tmp = *a;
6     *a = *b;
7     *b = tmp;
8 }
```

## LINE BY LINE

- 1 `int *a` is "the address of an integer". The `*` is part of the type.
- 3 The temporary variable. Without it, the swap is impossible.
- 5 Sets aside the value of `a`.
- 6 Puts the value of `b` into `a`. At this point the original value of `a` only exists in `tmp`.
- 7 Puts the saved value into `b`. Swap done.

To test it, whoever calls it writes `ft_swap(&x, &y)` — the `&` is what turns a variable into its address.

## L1 rev\_print

Show the string backwards.

### WHAT THE SUBJECT ASKS FOR

Show the received string backwards, followed by a newline. If the number of parameters is not 1, show only the newline.

Allowed functions: `write`.

```
$> ./rev_print "dub0 a POIL" | cat -e
LIOP a 0bud$
$> ./rev_print | cat -e
$
```

### HOW TO THINK ABOUT IT

- 1 To write it backwards you have to start at the end, and to reach the end you first have to measure the string.
- 2 Measuring is `ft_strlen` done by hand: walk until the `'\0'`.
- 3 At that point the index sits **right on top of** the `'\0'`, which you don't want to write. So you step back one.
- 4 Then you walk backwards until you pass index 0.

### THE CODE

rev\_print.c

```
1 #include <unistd.h>
2
3 int main(int argc, char **argv)
4 {
5     int i;
6
7     if (argc == 2)
8     {
9         i = 0;
10        while (argv[1][i])
11            i++;
12        i--;
13        while (i >= 0)
14        {
15            write(1, &argv[1][i], 1);
16            i--;
17        }
18    }
19    write(1, "\n", 1);
20    return (0);
21 }
```

### LINE BY LINE

- |       |                                                                                                                                 |
|-------|---------------------------------------------------------------------------------------------------------------------------------|
| 7     | "Exactly 1 argument" is <code>argc == 2</code> , because <code>argv[0]</code> is the program's name.                            |
| 10-11 | Measures the string: walks up to the <code>'\0'</code> . At the end, <code>i</code> is the length.                              |
| 12    | <b>Step back one.</b> Without this line the first character out would be the <code>'\0'</code> , and the output would be wrong. |
| 13    | Walks backwards while still inside the string. When <code>i</code> reaches <code>-1</code> , it stops.                          |
| 15    | Writes character number <code>i</code> . The <code>&amp;</code> gives its address, which is what <code>write</code> wants.      |
| 19    | The newline <b>outside</b> the <code>if</code> : this way it comes out both in the normal case and in the no-arguments case.    |

**WHAT THE SUBJECT ASKS FOR**

Show the string with the case of every letter flipped: lowercase becomes uppercase and uppercase becomes lowercase. Anything that isn't a letter stays the same. A newline at the end.

If the number of arguments is not 1, show only the newline.

Allowed functions: `write`.

```
$> ./ulstr "3:21 Ba tOut moUn ki Ka di KE" | cat -e
3:21 bA ToUT MOuN KI kA DI ke$
```

**HOW TO THINK ABOUT IT**

- 1 This is **the** pattern of the exam: walk the string and, for each character, decide what to write. Several exercises ahead are this exact loop with two different lines inside it.
- 2 The decision here: if it's lowercase, go up; if it's uppercase, go down; otherwise, leave it alone.
- 3 Uppercase letters sit exactly 32 below lowercase letters in the character table, so the conversion is a single addition or subtraction.

**THE CODE****ulstr.c**

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     main(int argc, char **argv)
9  {
10     int    i;
11     char   c;
12
13     if (argc == 2)
14     {
15         i = 0;
16         while (argv[1][i])
17         {
18             c = argv[1][i];
19             if (c >= 'a' && c <= 'z')
20                 c = c - 32;
21             else if (c >= 'A' && c <= 'Z')
22                 c = c + 32;
23             ft_putchar(c);
24             i++;
25         }
26     }
27     ft_putchar('\n');
28     return (0);
29 }
```

## LINE BY LINE

- 10–11** Two variables: the index walking the string, and a copy of the current character so it can be modified.
- 16** The usual loop: walks up to the `'\0'`.
- 18** Takes a copy of the character. You work on the copy, not on the original argument.
- 19–20** If it's between `'a'` and `'z'`, it's lowercase, so `- 32` moves it up to uppercase.
- 21–22** Otherwise, if it's between `'A'` and `'Z'`, it's uppercase, so `+ 32` moves it down to lowercase.
- 23** Writes the character, changed or not. Anything that isn't a letter reaches here without going through either `if`.
- 27** The final newline, outside the `argc` check.

The `else if` on line 21 is mandatory. With two separate `if` statements, a lowercase letter would turn uppercase in the first one and flip back to lowercase in the second — the program would do nothing.

## L1 rotone

Each letter moves to the next one in the alphabet.

### WHAT THE SUBJECT ASKS FOR

Show the string with each letter replaced by the next one in the alphabet. `z` becomes `a` and `Z` becomes `A`. Case doesn't change and anything that isn't a letter stays the same. A newline at the end.

If the number of arguments is not 1, show only the newline.

Allowed functions: `write`.

```
$> ./rotone "AkjhZ zLKIJz , 23y " | cat -e
BlkiA aMLJKa , 23z $
```

### HOW TO THINK ABOUT IT

- 1 It's the same loop as `ulstr`, only the decision inside changes.
- 2 Adding 1 solves almost everything, **except** at the end of the alphabet: `'z' + 1` would give `'{'`, which isn't a letter at all.
- 3 So the two edge cases are handled first, and only then the general case.

## THE CODE

rotone.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     main(int argc, char **argv)
9  {
10     int    i;
11     char   c;
12
13     if (argc == 2)
14     {
15         i = 0;
16         while (argv[1][i])
17         {
18             c = argv[1][i];
19             if (c == 'z')
20                 c = 'a';
21             else if (c == 'Z')
22                 c = 'A';
23             else if ((c >= 'a' && c < 'z') || (c >= 'A' && c < 'Z'))
24                 c = c + 1;
25             ft_putchar(c);
26             i++;
27         }
28     }
29     ft_putchar('\n');
30     return (0);
31 }
```

## LINE BY LINE

19–20 The special case at the end of the lowercase alphabet: `z` wraps back to the start.

21–22 The same for uppercase.

23 Only whoever isn't `z` or `Z` reaches here. The test uses `<`, not `<=`, on purpose: `z` has already been handled.

24 The general case: the next letter is the one with the code right above.

There's an alternative with no special cases at all, using the remainder formula: `c = 'a' + (c - 'a' + 1) % 26;` for lowercase and the equivalent for uppercase. It's shorter, and it's mandatory for `rot_13` — there the jump is 13, and the special cases would be 13 on each side.

## L1 rot\_13

Each letter moves forward 13 positions, wrapping around.

### WHAT THE SUBJECT ASKS FOR

Show the string with each letter replaced by the letter 13 positions ahead in the alphabet. `z` becomes `m` and `Z` becomes `M`. Case doesn't change. A newline at the end.

If the number of arguments is not 1, show only the newline.

Allowed functions: `write`.

```
$> ./rot_13 "My horse is Amazing." | cat -e
Zl ubefr vf Nznmvat.$
```

## HOW TO THINK ABOUT IT

- 1 It's `rotone` with 13 instead of 1, but now the special cases would be 13 letters on each side — not practical.
- 2 The solution is the formula that wraps around the alphabet on its own. Read it in three steps: put the letter on a 0-to-25 scale, move 13, and if it goes past 25, wrap around with the remainder of dividing by 26. At the end, put it back on the letter scale.

## THE CODE

`rot_13.c`

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     main(int argc, char **argv)
9  {
10     int    i;
11     char   c;
12
13     if (argc == 2)
14     {
15         i = 0;
16         while (argv[1][i])
17         {
18             c = argv[1][i];
19             if (c >= 'a' && c <= 'z')
20                 c = 'a' + (c - 'a' + 13) % 26;
21             else if (c >= 'A' && c <= 'Z')
22                 c = 'A' + (c - 'A' + 13) % 26;
23             ft_putchar(c);
24             i++;
25         }
26     }
27     ft_putchar('\n');
28     return (0);
29 }
```

## LINE BY LINE

- 20 `c - 'a'` puts the letter on the 0-to-25 scale. Add 13. The `% 26` is what wraps it around: 25 + 13 is 38, and 38 divided by 26 leaves a remainder of 12. Finally, `'a' +` puts it back on the letter scale.
- 22 Exactly the same for uppercase, just changing the starting point from `'a'` to `'A'`.

This formula works for any jump size. Swap the 13 for a 1 and you get `rotone`; the structure of the exercise is the same.

## L1 repeat\_alpha

Each letter repeats as many times as its position in the alphabet.

## WHAT THE SUBJECT ASKS FOR

Show the string, repeating each letter as many times as its alphabet index: `a` comes out once, `b` twice, `e` five times, and so on. Case doesn't change and anything that isn't a letter comes out once. A newline at the end.

If the number of arguments is not 1, show only the newline.

Allowed functions: `write`.

```
$> ./repeat_alpha "abacadaba 42!" | cat -e
abbaccaddddabba 42!$
```

## HOW TO THINK ABOUT IT

- 1 The usual loop, but now each pass writes a variable number of characters — so you need a **second loop nested inside**.
- 2 How many times? For letters, the position in the alphabet plus one (`a` is position 0, but comes out once). For everything else, once.
- 3 The organizing trick: start by assuming the normal case (`n = 1`) and only correct it if it's a letter. That way spaces and digits are handled without writing anything special for them.

## THE CODE

repeat\_alpha.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int main(int argc, char **argv)
9 {
10     int i;
11     int j;
12     int n;
13     char c;
14
15     if (argc == 2)
16     {
17         i = 0;
18         while (argv[1][i])
19         {
20             c = argv[1][i];
21             n = 1;
22             if (c >= 'a' && c <= 'z')
23                 n = c - 'a' + 1;
24             else if (c >= 'A' && c <= 'Z')
25                 n = c - 'A' + 1;
26             j = 0;
27             while (j++ < n)
28                 ft_putchar(c);
29             i++;
30         }
31     }
32     ft_putchar('\n');
33     return (0);
34 }
```

## LINE BY LINE

10–13 Four variables: the outer index, the inner counter, how many times to repeat, and the current character.

21 The normal case: once. Everything that isn't a letter keeps this value.

22–23 If it's lowercase, the repeat count is its alphabet position plus one: `a` gives 1, `b` gives 2.

24–25 Same for uppercase.

26–28 The inner loop: writes the character `n` times. `j++ < n` compares first and increments after, so it runs exactly `n` times.

29 Moves to the next character of the string — this `i++` belongs to the outer loop.

### WHAT THE SUBJECT ASKS FOR

The program receives three arguments: a string, the letter to search for, and the letter to put in its place. It shows the string with every occurrence replaced, followed by a newline.

If the number of arguments is not 3, it shows only the newline. The second and third arguments must be **a single letter** — if they're longer, it also shows only the newline.

Allowed functions: `write`, `exit`.

```
$> ./search_and_replace "Papache est un sabre" "a" "o" | cat -e
Popoche est un sobre$
$> ./search_and_replace "zaz" "art" "zul" | cat -e
$
$> ./search_and_replace "zaz" "r" "u" | cat -e
zaz$
```

### HOW TO THINK ABOUT IT

- 1 The replacement itself is trivial: walk the string and, if the character matches the one you're searching for, write the other one instead.
- 2 The difficulty is in the **validation**. The second example only gives a newline because `"art"` has three letters, and it's easy to miss.
- 3 To know if an argument is a single letter, measure its length — that's why `ft_strlen` shows up in here.
- 4 If the letter you're searching for doesn't exist in the string, nothing special needs to happen: the loop writes everything unchanged and the result comes out right on its own.

## THE CODE

search\_and\_replace.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int ft_strlen(char *s)
9 {
10     int i;
11
12     i = 0;
13     while (s[i])
14         i++;
15     return (i);
16 }
17
18 int main(int argc, char **argv)
19 {
20     int i;
21
22     if (argc == 4 && ft_strlen(argv[2]) == 1 && ft_strlen(argv[3]) == 1)
23     {
24         i = 0;
25         while (argv[1][i])
26         {
27             if (argv[1][i] == argv[2][0])
28                 ft_putchar(argv[3][0]);
29             else
30                 ft_putchar(argv[1][i]);
31             i++;
32         }
33     }
34     ft_putchar('\n');
35     return (0);
36 }
```

## LINE BY LINE

- 8-16 `ft_strlen` from the previous exercise, copied in here. It's only used to validate the arguments.
- 
- 22 All three conditions at once: three arguments (so `argc == 4`), and the second and third being single letters.
- 
- 25 The loop walks the **first** string, which is the one being shown.
- 
- 27 `argv[2][0]` is the first (and only) character of the second argument — the letter to search for.
- 
- 28 If it matches, the replacement letter is written instead of the original.
- 
- 30 Otherwise, the character is written exactly as it was.
- 
- 34 The newline outside the `if`, as always: it covers every invalid case in one go.

The subject allows `exit`, but none is needed: putting the newline outside the `if` already handles every error case.

## L1 first\_word

Show the first word of the string.

### WHAT THE SUBJECT ASKS FOR

Show the first word of the string, followed by a newline. A word is a chunk of the string delimited by spaces or tabs, or by the start and end of the string.

If the number of parameters is not 1, or if there is no word at all, show only the newline.

Allowed functions: `write`.

```
$> ./first_word " lorem,ipsum " | cat -e
lorem,ipsum$
$> ./first_word " " | cat -e
$
$> ./first_word "a" "b" | cat -e
$
```

### HOW TO THINK ABOUT IT

- 1 This starts the family of word exercises, six of them in the pool. They all depend on the same question: "is this character a separator?". It's worth writing that as a standalone function.
- 2 The pattern is always the same: **skip the separators**, then **handle the word**.
- 3 If the string is only spaces, the first loop reaches `'\0'` and the second one writes nothing — the case resolves itself for free.

### THE CODE

first\_word.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int is_space(char c)
9 {
10     return (c == ' ' || c == '\t');
11 }
12
13 int main(int argc, char **argv)
14 {
15     int i;
16
17     if (argc == 2)
18     {
19         i = 0;
20         while (is_space(argv[1][i]))
21             i++;
22         while (argv[1][i] && !is_space(argv[1][i]))
23         {
24             ft_putchar(argv[1][i]);
25             i++;
26         }
27     }
28     ft_putchar('\n');
29     return (0);
30 }
```

## LINE BY LINE

- 8–11** The helper. Returns true if the character is a space or a tab. **The tab counts** — whoever only handles the space passes the visible examples and fails the grader's test.
- 
- 20–21** First loop: skips the separators at the start. Notice it doesn't need to test for `'\0'`, because `'\0'` is neither a space nor a tab and the loop stops there anyway.
- 
- 22** Second loop: writes while there are characters **and** it's not a separator. Both conditions are needed: one stops at the end of the string, the other at the end of the word.
- 
- 28** The newline outside the `if`.
- 

## L1 **fizzbuzz**

The numbers from 1 to 100, with `fizz`, `buzz` and `fizzbuzz`.

### WHAT THE SUBJECT ASKS FOR

Show the numbers from 1 to 100, one per line. If the number is a multiple of 3, write `fizz` instead; if it's a multiple of 5, `buzz`; if it's a multiple of both, `fizzbuzz`.

Allowed functions: `write`.

```
$> ./fizzbuzz | head -6
1
2
fizz
4
buzz
fizz
```

### HOW TO THINK ABOUT IT

- ❶ **The order of the tests is the whole exercise.** The double case has to be tested first: if you test the multiple of 3 first, 15 comes out as `fizz` and the program fails.
- ❷ "A multiple of 3 and of 5 at the same time" is the same as "a multiple of 15".
- ❸ There's no `printf`, so you need the function that writes numbers — `ft_putnbr` from the basics.

## THE CODE

fizzbuzz.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  void    ft_putnbr(int n)
9  {
10     if (n >= 10)
11         ft_putnbr(n / 10);
12     ft_putchar(n % 10 + '0');
13 }
14
15 int     main(void)
16 {
17     int i;
18
19     i = 1;
20     while (i <= 100)
21     {
22         if (i % 15 == 0)
23             write(1, "fizzbuzz", 8);
24         else if (i % 3 == 0)
25             write(1, "fizz", 4);
26         else if (i % 5 == 0)
27             write(1, "buzz", 4);
28         else
29             ft_putnbr(i);
30         ft_putchar('\n');
31         i++;
32     }
33     return (0);
34 }
```

## LINE BY LINE

**8-13** `ft_putnbr`. It's recursive: with 42, it calls itself with 4, that call writes `'4'`, and on the way back it writes `'2'`. That's how the digits come out in the right order.

**12** `n % 10` is the last digit, and adding `'0'` turns the number into the matching character.

**22** The double case, tested **first**. This is the line that decides whether the exercise passes.

**24-27** The two simple cases, in either order relative to each other.

**29** Only whoever is neither a multiple of 3 nor of 5 reaches here.

**30** The newline after each line, whichever case it was.

## L2 Level 2

15 exercises. Comparisons between two strings, bits, and converting text into numbers.

### L2 alpha\_mirror

Each letter swaps with its mirror in the alphabet.

#### WHAT THE SUBJECT ASKS FOR

Show the string with each letter replaced by the opposite letter in the alphabet: **a** becomes **z**, **b** becomes **y**, **M** becomes **N**. Case doesn't change and anything that isn't a letter stays the same. A newline at the end.

If the number of arguments is not 1, show only the newline.

Allowed functions: `write`.

```
$> ./alpha_mirror "My horse is Amazing." | cat -e
Nb slihv rh ZnzarMt.$
```

#### HOW TO THINK ABOUT IT

- 1 Once again the same loop as `ulstr`, with a different decision inside.
- 2 The idea: `c - 'a'` tells you how far the letter is from the start of the alphabet. The mirror letter is **the same distance, but counted from the end**.
- 3 You don't need to know the alphabet has 26 letters, or that **a** is the first one: the subtraction takes care of that by itself.

#### THE CODE

alpha\_mirror.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int main(int argc, char **argv)
9 {
10     int i;
11     char c;
12
13     if (argc == 2)
14     {
15         i = 0;
16         while (argv[1][i])
17         {
18             c = argv[1][i];
19             if (c >= 'a' && c <= 'z')
20                 c = 'z' - (c - 'a');
21             else if (c >= 'A' && c <= 'Z')
22                 c = 'Z' - (c - 'A');
23             ft_putchar(c);
24             i++;
25         }
26     }
27     ft_putchar('\n');
28     return (0);
29 }
```

## LINE BY LINE

- 20 `c - 'a'` is the distance to `a`. `'z' -` that distance gives the letter the same distance from `z`. With `c = 'a'` you get `'z'`; with `c = 'b'` you get `'y'`.
- 22 The same for uppercase, starting from `'Z'`.

## L2 ft\_strrev

Function that reverses a string in place.

### WHAT THE SUBJECT ASKS FOR

Write a function that reverses a string **in place** and returns the parameter it received.

The declaration is `char *ft_strrev(char *str);`

"In place" means you can't create a new string: you have to work on this one.

### HOW TO THINK ABOUT IT

- 1 If you can't create new space, the only way is to **swap pairs of characters**: the first with the last, the second with the second-to-last, and so on.
- 2 Two indices, one at each end, moving toward each other.
- 3 When do they stop? When they cross. If they kept going, they'd start undoing the swaps they already made.
- 4 Each swap needs a temporary variable, just like in `ft_swap`.

### THE CODE

ft\_strrev.c

```
1 char *ft_strrev(char *str)
2 {
3     int i;
4     int j;
5     char tmp;
6
7     i = 0;
8     j = 0;
9     while (str[j])
10         j++;
11     j--;
12     while (i < j)
13     {
14         tmp = str[i];
15         str[i] = str[j];
16         str[j] = tmp;
17         i++;
18         j--;
19     }
20     return (str);
21 }
```

## LINE BY LINE

- 9-10 Measures the string: `j` ends up sitting on the `'\0'`.
- 11 Steps back one, so `j` lands on the last real character.
- 12 While the two indices haven't crossed. If the string has an odd number of characters, the middle one stays where it is — and that's correct.
- 14-16 The classic three-step swap, with `tmp` holding one of the values.
- 17-18 The two ends move toward each other.
- 20 Returns the same pointer it received, as the subject asks.

## L2 ft\_strcmp

Function that compares two strings.

### WHAT THE SUBJECT ASKS FOR

Reproduce the behaviour of `strcmp` from the man page: returns 0 if the strings are equal, a negative number if the first one comes earlier alphabetically, and a positive one if it comes later.

The declaration is `int ft_strcmp(char *s1, char *s2);`

### HOW TO THINK ABOUT IT

- 1 Advance while the characters are equal. Wherever the loop stops is the first difference.
- 2 If the strings are equal all the way to the end, the loop stops at the `'\0'` of both, and the difference comes out zero — the right answer, with no extra code.
- 3 The sign comes from the subtraction: if the `s1` character is smaller, the subtraction is negative.

### THE CODE

ft\_strcmp.c

```
1 int ft_strcmp(char *s1, char *s2)
2 {
3     int i;
4
5     i = 0;
6     while (s1[i] && s1[i] == s2[i])
7         i++;
8     return ((unsigned char)s1[i] - (unsigned char)s2[i]);
9 }
```

### LINE BY LINE

- 6 Two conditions at once: hasn't reached the end of `s1` **and** the characters are equal. Either one failing stops the loop.
- 8 The subtraction at the position where they stopped. The `(unsigned char)` is for accented characters or ones with the high bit set: without it, a character with a code above 127 would be read as negative and the sign would come out flipped.

This version returns the actual difference between the two characters, for example `-99`. The system's `strcmp` might return `-1`. That's fine: what gets compared is the **sign**, not the value.

## L2 max

Function that returns the largest integer in an array.

### WHAT THE SUBJECT ASKS FOR

Write the function `int max(int* tab, unsigned int len);`

The first parameter is an array of integers and the second is how many elements it has. The function returns the largest one. If the array is empty, it returns 0.

### HOW TO THINK ABOUT IT

- 1 Keep a "best so far" and compare it against each element.
- 2 **What value do you start with?** It can't be 0: an array with only negative numbers would return 0, which isn't even in it. Start with the array's first element.
- 3 Since the first one has already been used, the loop starts at the second.
- 4 The empty-array case has to be handled before anything else, or `tab[0]` reads memory that doesn't exist.

## THE CODE

max.c

```
1 int max(int *tab, unsigned int len)
2 {
3     unsigned int i;
4     int best;
5
6     if (len == 0)
7         return (0);
8     best = tab[0];
9     i = 1;
10    while (i < len)
11    {
12        if (tab[i] > best)
13            best = tab[i];
14        i++;
15    }
16    return (best);
17 }
```

## LINE BY LINE

- 3 `i` is declared `unsigned int` on purpose, matching `len`. Comparing an `int` to an `unsigned int` triggers a compiler warning, and a warning is a failed exercise.
- 6-7 The empty-array case, handled first so `tab[0]` on the next line is safe.
- 8 The candidate starts at the first element, not at zero.
- 9 The loop starts at the **second** element, because the first is already the candidate.
- 12-13 If a larger one is found, it becomes the new candidate.
- 16 At the end, the candidate is the largest of all.

## L2 ft\_strdup

Function that returns a fresh copy of a string.

### WHAT THE SUBJECT ASKS FOR

Reproduce the behaviour of `strdup` from the man page: returns a new copy of the received string.

The declaration is `char *ft_strdup(char *src);`

Allowed functions: `malloc`.

### HOW TO THINK ABOUT IT

- ❶ Here you need to return a string that doesn't exist yet. A normal variable declared inside the function disappears when the function ends, so you request memory with `malloc`.
- ❷ The `malloc` pattern always has three steps, in this order: **count**, **request**, **fill in**.
- ❸ In the counting step, the number is the length **plus one**, to fit the `'\0'` at the end. Forgetting that `+ 1` is the most common mistake in the whole of level 2.

## THE CODE

ft\_strdup.c

```
1 #include <stdlib.h>
2
3 char *ft_strdup(char *src)
4 {
5     char *dup;
6     int i;
7
8     i = 0;
9     while (src[i])
10         i++;
11     dup = malloc(sizeof(char) * (i + 1));
12     if (!dup)
13         return (0);
14     i = 0;
15     while (src[i])
16     {
17         dup[i] = src[i];
18         i++;
19     }
20     dup[i] = '\0';
21     return (dup);
22 }
```

## LINE BY LINE

|       |                                                                                                                                                                                    |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | <code>malloc</code> lives in <code>stdlib.h</code> .                                                                                                                               |
| 8-10  | Step 1, count: measures the source string.                                                                                                                                         |
| 11    | Step 2, request. <code>sizeof(char)</code> is how much space one character takes, multiplied by how many are needed. The <code>+ 1</code> is the space for the <code>'\0'</code> . |
| 12-13 | If <code>malloc</code> fails it returns <code>0</code> , and you also need to return <code>0</code> instead of continuing to write into memory that doesn't exist.                 |
| 14    | <code>i</code> is reused, starting back at zero.                                                                                                                                   |
| 15-19 | Step 3, fill in: copies character by character.                                                                                                                                    |
| 20    | The loop stopped at the <code>'\0'</code> without copying it, just like in <code>ft_strcpy</code> . It's added by hand.                                                            |
| 21    | Returns the address of the new copy.                                                                                                                                               |

## L2 ft\_atoi

Function that converts text into an integer.

### WHAT THE SUBJECT ASKS FOR

Write a function that converts the received string into an integer and returns it, like `atoi` from the man page.

The declaration is `int ft_atoi(const char *str);`

`atoi` skips leading spaces, reads a sign if there is one, reads the digits, and stops at the first character that isn't a digit.

### HOW TO THINK ABOUT IT

- 1 The recipe for turning text into a number is always the same: start at 0 and, for each digit, **multiply what you have so far by 10 and add the new one**. With `"123"`:  $0 \times 10 + 1 = 1$ ;  $1 \times 10 + 2 = 12$ ;  $12 \times 10 + 3 = 123$ .
- 2 The work splits into three clear phases, and writing one loop for each is what makes this easy: skip spaces, read the sign, read the digits.
- 3 `c - '0'` turns the character `'7'` into the number 7, because digits are consecutive codes.

## THE CODE

ft\_atoi.c

```
1 int ft_atoi(const char *str)
2 {
3     int i;
4     int sign;
5     int res;
6
7     i = 0;
8     sign = 1;
9     res = 0;
10    while (str[i] == ' ' || (str[i] >= 9 && str[i] <= 13))
11        i++;
12    while (str[i] == '+' || str[i] == '-')
13    {
14        if (str[i] == '-')
15            sign = -sign;
16        i++;
17    }
18    while (str[i] >= '0' && str[i] <= '9')
19    {
20        res = res * 10 + (str[i] - '0');
21        i++;
22    }
23    return (res * sign);
24 }
```

## LINE BY LINE

- 8 The sign starts positive. At the end, the result is multiplied by it.
- 10 Phase 1: skip spaces. `atoi`'s notion of "space" includes tab and newline, which are codes 9 through 13 — hence this second test.
- 12–17 Phase 2: signs. Each `-` flips the accumulated sign, so `--42` gives 42.
- 18 Phase 3: digits. The loop stops by itself at the first character that isn't a digit, which is why `"-42abc"` gives `-42`.
- 20 The times-ten recipe. The parentheses aren't strictly needed, but they help readability.
- 23 The sign is applied only at the end.

This version accepts several signs in a row, by Piscine tradition. The real `atoi` from the man page only accepts one — if you need to match the man page exactly, swap the `while` on line 12 for an `if`.

## L2 do\_op

Do a calculation with two numbers and an operator.

### WHAT THE SUBJECT ASKS FOR

The program receives three arguments: a number, an operator (`+`, `-`, `*`, `/` or `%`) and another number. It shows the result of the calculation followed by a newline.

If the number of parameters is not 3, it shows only the newline.

Allowed functions: `atoi`, `printf`, `write`.

```
$> ./do_op "1" "+" "-43" | cat -e
-42$
$> ./do_op "9828" "/" 234 | cat -e
42$
```

## HOW TO THINK ABOUT IT

- 1 **Look at the `Allowed functions` line first.** This exercise allows the real `atoi` and `printf`, which removes half the work. It's one of the fastest in level 2.
- 2 Convert the two numbers, check which operator it is, do the calculation.
- 3 The operator arrives as a **one-character string**. You compare `argv[2][0]`, which is the character, not `argv[2]`, which is the string.

## THE CODE

do\_op.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char **argv)
5 {
6     int a;
7     int b;
8
9     if (argc != 4)
10    {
11        printf("\n");
12        return (0);
13    }
14    a = atoi(argv[1]);
15    b = atoi(argv[3]);
16    if (argv[2][0] == '+')
17        printf("%d\n", a + b);
18    else if (argv[2][0] == '-')
19        printf("%d\n", a - b);
20    else if (argv[2][0] == '*')
21        printf("%d\n", a * b);
22    else if (argv[2][0] == '/')
23        printf("%d\n", a / b);
24    else if (argv[2][0] == '%')
25        printf("%d\n", a % b);
26    return (0);
27 }
```

## LINE BY LINE

- 9–13 The wrong-arguments case, resolved right away, and the program ends there.
- 14–15 `atoi` does the previous exercise's job for free. Note the second number is `argv[3]`, since `argv[2]` is the operator.
- 16 `argv[2][0]` is the first character of the second argument. `'+'` with single quotes is a character; `"+"` with double quotes would be a string, and in C you don't compare strings with `==`.
- 17 `%d` in `printf` means "write an integer here". The `\n` is the newline the subject asks for.

Some versions of this subject ask for `Stop : division by zero` and `Stop : modulo by zero` when the second number is zero. **Read your own subject.** If it asks for that, add a test for `b == 0` before the divisions that writes the message and returns.

## L2 last\_word

Show the last word of the string.

## WHAT THE SUBJECT ASKS FOR

Show the last word of the string, followed by a newline. A word is a chunk of the string delimited by spaces or tabs, or by the start and end of the string.

If the number of parameters is not 1, or if there are no words, show only the newline.

Allowed functions: `write`.

```
$> ./last_word "this          ...          is sparta, then again, maybe    not" | cat -e
not$
$> ./last_word "  lorem,ipsum  " | cat -e
lorem,ipsum$
```

## HOW TO THINK ABOUT IT

- 1 It's `first_word` backwards, with one extra difficulty: to write the word you need to know where it **starts** and where it **ends**, and walking from the end you find the end first.
- 2 Four steps: go to the end of the string; step back while it's a separator (skips the trailing spaces); mark the end of the word there; keep stepping back while it's **not** a separator.
- 3 When that last loop stops, the index sits one character **before** the word. So you step forward one before writing.
- 4 The `i >= 0` test in every loop is what keeps you from reading outside the string when it's empty or only spaces.

## THE CODE

`last_word.c`

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     is_space(char c)
9  {
10     return (c == ' ' || c == '\t');
11 }
12
13 int     main(int argc, char **argv)
14 {
15     int i;
16     int end;
17
18     if (argc == 2)
19     {
20         i = 0;
21         while (argv[1][i])
22             i++;
23         i--;
24         while (i >= 0 && is_space(argv[1][i]))
25             i--;
26         end = i;
27         while (i >= 0 && !is_space(argv[1][i]))
28             i--;
29         i++;
30         while (i <= end)
31         {
32             ft_putchar(argv[1][i]);
33             i++;
34         }
35     }
36     ft_putchar('\n');
37     return (0);
38 }
```

## LINE BY LINE

- 21–23 Goes to the end and steps back one, landing on the last real character.
- 24–25 Skips the trailing separators. This is where the `" lorem, ipsum "` case gets solved.
- 26 Marks the end of the word.
- 27–28 Steps back while it's **not** a separator: this walks back over the word.
- 29 Steps forward one. Now `i` sits on the word's first character.
- 30–34 Writes from the start to the end of the word, now moving forward.

If the string is empty, `i` ends up at `-1` right on line 23, every loop is skipped, `end` stays at `-1`, and the writing loop never runs. Only the newline comes out, which is what the subject asks for.

## L2 inter

The characters that appear in both strings, with no duplicates.

### WHAT THE SUBJECT ASKS FOR

Show the characters that appear in both strings, with no repeats, in the order they appear in the first one. A newline at the end.

If the number of arguments is not 2, show only the newline.

Allowed functions: `write`.

```
$> ./inter "padinton" "paqefwtdjetyiytjneytjoeyjnejejj" | cat -e
padinto$
```

### HOW TO THINK ABOUT IT

- Two questions for each character of the first string: **does it exist in the second one?** and **have I already written this one?**
- The first is solved with a loop that searches the second string.
- The second is solved with a new idea: an array of 256 markers, one for every possible character. Since each character is a number between 0 and 255, you can use the character itself as a **position** in the array. `a`'s slot is `seen['a']`.
- Mark it 1 when you write it, and before writing check whether the slot is already marked.

## THE CODE

inter.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     main(int argc, char **argv)
9  {
10     int    i;
11     int    j;
12     char    seen[256];
13
14     i = 0;
15     while (i < 256)
16         seen[i++] = 0;
17     if (argc == 3)
18     {
19         i = 0;
20         while (argv[1][i])
21         {
22             j = 0;
23             while (argv[2][j] && argv[2][j] != argv[1][i])
24                 j++;
25             if (argv[2][j] && !seen[(unsigned char)argv[1][i]])
26             {
27                 ft_putchar(argv[1][i]);
28                 seen[(unsigned char)argv[1][i]] = 1;
29             }
30             i++;
31         }
32     }
33     ft_putchar('\n');
34     return (0);
35 }
```

### LINE BY LINE

- 12** The marker array. 256 because that's how many different characters fit in a byte.
- 14–16** Sets everything to zero. A freshly created array has garbage inside, not zeros.
- 22–24** Searches for the character in the second string. The loop stops either right on it (found it) or at the `'\0'` (didn't find it).
- 25** Two conditions: `argv[2][j]` still exists, so it was found; and the slot isn't marked yet, so it's the first time.
- 27–28** Writes it and marks the slot, so the same character doesn't get written again.

The `(unsigned char)` around the index is mandatory. Without it, an accented character would be read as a negative number, and `seen[-30]` reads memory outside the array — the program crashes.

## L2 union

The characters that appear in one string or the other, with no duplicates.

### WHAT THE SUBJECT ASKS FOR

Show the characters that appear in one of the two strings or the other, with no repeats, in the order they appear on the command line. A newline at the end.

If the number of arguments is not 2, show only the newline.

Allowed functions: `write`.

```
$> ./union zpadinton "paqefwtdjetyiytjneytjoeyjnejejj" | cat -e
zpadintoqefwjy$
```

## HOW TO THINK ABOUT IT

- 1 It's `inter` without the search through the second string, but with an outer loop that passes over both.
- 2 The markers do all the work: write any character that hasn't been written yet, wherever it comes from.
- 3 It's worth practising this one and `inter` on the same day: whoever solves one gets the other almost for free.

## THE CODE

union.c

```
1  #include <unistd.h>
2
3  void  ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int   main(int argc, char **argv)
9  {
10     int    a;
11     int    i;
12     char    seen[256];
13
14     i = 0;
15     while (i < 256)
16         seen[i++] = 0;
17     if (argc == 3)
18     {
19         a = 1;
20         while (a <= 2)
21         {
22             i = 0;
23             while (argv[a][i])
24             {
25                 if (!seen[(unsigned char)argv[a][i]])
26                 {
27                     ft_putchar(argv[a][i]);
28                     seen[(unsigned char)argv[a][i]] = 1;
29                 }
30                 i++;
31             }
32             a++;
33         }
34     }
35     ft_putchar('\n');
36     return (0);
37 }
```

## LINE BY LINE

- |       |                                                                                                                                             |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 19-20 | The new loop: <code>a</code> is 1 on the first pass and 2 on the second, i.e. it walks <code>argv[1]</code> and then <code>argv[2]</code> . |
| 23    | The same usual loop, but now over <code>argv[a]</code> , which changes on each pass of the outer one.                                       |
| 25    | Just the question "have I already written this one?". No search through the other string.                                                   |
| 32    | Moves from the first string to the second.                                                                                                  |

**WHAT THE SUBJECT ASKS FOR**

Check whether it's possible to spell the first string using characters from the second one, respecting the order they appear in it. If it's possible, show the first string followed by a newline; otherwise, only the newline.

If the number of arguments is not 2, show only the newline.

Allowed functions: `write`.

```
$> ./wdmatch "faya" "fgvvfdxcacpolhyghbreda" | cat -e
faya$
$> ./wdmatch "faya" "fgvvfdxcacpolhyghbred" | cat -e
$
```

**HOW TO THINK ABOUT IT**

- 1 A new technique, but a simple one: **one index in each string**.
- 2 The index of the second string moves **always**, character by character. The one for the first only advances when it finds the character it's looking for.
- 3 At the end, if the first string's index has reached `'\0'`, every character was found in the right order.
- 4 No marking or backward searching needed: one pass over each string is enough.

**THE CODE**`wdmatch.c`

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int main(int argc, char **argv)
9 {
10     int i;
11     int j;
12
13     if (argc == 3)
14     {
15         i = 0;
16         j = 0;
17         while (argv[1][i] && argv[2][j])
18         {
19             if (argv[1][i] == argv[2][j])
20                 i++;
21             j++;
22         }
23         if (!argv[1][i])
24         {
25             j = 0;
26             while (argv[1][j])
27                 ft_putchar(argv[1][j++]);
28         }
29     }
30     ft_putchar('\n');
31     return (0);
32 }
```

## LINE BY LINE

- 17 The loop walks while there are characters in both. Either one ending stops it.
- 19–20 The first string's index only advances when there's a match.
- 21 The second one's always advances, match or not.
- 23 `!argv[1][i]` means "`i` is sitting on the `'\0'`", i.e. the whole first string has been walked.
- 25–27 Writes the first string. The `j++` inside the brackets uses the value and only then increments.

## L2 `is_power_of_2`

Function that says whether a number is a power of 2.

### WHAT THE SUBJECT ASKS FOR

Write a function that determines whether a number is a power of 2. Returns 1 if it is, 0 if it isn't.

The declaration is `int is_power_of_2(unsigned int n);`

### HOW TO THINK ABOUT IT

- 1 In binary, a power of 2 has **exactly one bit set to 1**: 8 is `1000`, 16 is `10000`.
- 2 Subtracting 1 from one of those numbers flips every bit below it: `8 - 1` gives `0111`.
- 3 So the bitwise `AND` of the two (`n & (n - 1)`) gives zero **only** for powers of 2.
- 4 Zero has to be handled separately: it isn't a power of 2, but `0 & (0-1)` would also give zero.

### THE CODE

`is_power_of_2.c`

```
1 int is_power_of_2(unsigned int n)
2 {
3     if (n == 0)
4         return (0);
5     return ((n & (n - 1)) == 0);
6 }
```

## LINE BY LINE

- 3–4 The zero case, handled first.
- 5 The test. `&` is bitwise "AND": it only leaves a 1 where both numbers have a 1. The `== 0` comparison returns 1 or 0, which is exactly what the subject asks for.

If this idea doesn't come to mind during the exam, the brute-force version also passes and takes three lines: divide by 2 while the number is even, then check whether what's left is 1. It's not worth losing time trying to be clever.

## L2 `print_bits`

Function that writes a byte in binary.

### WHAT THE SUBJECT ASKS FOR

Write a function that receives a byte and shows it in binary, **with no newline at the end**.

The declaration is `void print_bits(unsigned char octet);`

For example, the value 2 writes `00000010`.

Allowed functions: `write`.

## HOW TO THINK ABOUT IT

- 1 An `unsigned char` is 8 bits. You need to write all 8, from leftmost to rightmost.
- 2 To read bit number `i` there's a single recipe that always works: `(octet >> i) & 1`. The `>>` pushes the bits `i` places to the right, which puts the bit you want at the end; the `& 1` erases everything else.
- 3 Since you write from left to right, you start at bit 7 and count down to 0.
- 4 A bit is worth 0 or 1, which are **numbers**. Adding `'0'` turns them into the characters `'0'` and `'1'`.

## THE CODE

print\_bits.c

```
1 #include <unistd.h>
2
3 void print_bits(unsigned char octet)
4 {
5     int i;
6     char c;
7
8     i = 7;
9     while (i >= 0)
10    {
11        c = ((octet >> i) & 1) + '0';
12        write(1, &c, 1);
13        i--;
14    }
15 }
```

## LINE BY LINE

- |    |                                                                                                                         |
|----|-------------------------------------------------------------------------------------------------------------------------|
| 8  | Starts at the leftmost bit, which is number 7.                                                                          |
| 11 | The recipe: shifts by <code>i</code> places, keeps the last bit, and adds <code>'0'</code> to turn it into a character. |
| 12 | Writes that character.                                                                                                  |
| 13 | Steps down one bit.                                                                                                     |

The subject is explicit: **no newline at the end**. Adding one fails the exercise.

## L2 reverse\_bits

Function that reverses the bit order of a byte.

## WHAT THE SUBJECT ASKS FOR

Write a function that receives a byte, reverses its bits, and returns the result. For example, `0100 0001` becomes `1000 0010`.

The declaration is `unsigned char reverse_bits(unsigned char octet);`

## HOW TO THINK ABOUT IT

- 1 The result is built up from zero, one bit at a time.
- 2 Each pass: shift what you already have one place to the left (to open up room on the right) and put the new bit there.
- 3 Since the original bits are read from 0 upward and always land in the rightmost slot, they naturally come out in reverse order — exactly what's asked for.

## THE CODE

reverse\_bits.c

```
1 unsigned char reverse_bits(unsigned char octet)
2 {
3     unsigned char res;
4     int i;
5
6     res = 0;
7     i = 0;
8     while (i < 8)
9     {
10         res = (res << 1) | ((octet >> i) & 1);
11         i++;
12     }
13     return (res);
14 }
```

## LINE BY LINE

6 The result starts empty.

10 Two things happen in one line: `res << 1` opens up room on the right, and `|` (bitwise "OR") puts the bit read from the original there. `(octet >> i) & 1` is the same recipe from `print_bits`.

13 After eight passes, `res` has all the bits in reverse order.

## L2 swap\_bits

Function that swaps the two halves of a byte.

## WHAT THE SUBJECT ASKS FOR

Write a function that receives a byte, swaps its two halves, and returns the result. For example, `0100 0001` becomes `0001 0100`.

The declaration is `unsigned char swap_bits(unsigned char octet);`

## HOW TO THINK ABOUT IT

- 1 A byte has 8 bits, so each half has 4.
- 2 The left half has to move down 4 places and the right half up 4.
- 3 Then everything is joined with `|`.
- 4 Nothing needs to be masked off: whatever falls outside the 8 bits gets cut automatically because the return type is `unsigned char`.

## THE CODE

swap\_bits.c

```
1 unsigned char swap_bits(unsigned char octet)
2 {
3     return ((octet >> 4) | (octet << 4));
4 }
```

## LINE BY LINE

3 A single line. `octet >> 4` brings down the upper half; `octet << 4` brings up the lower half; `|` joins the two.

## L3 Level 3

15 exercises. Words, bases, math, the first memory requested with `malloc`, and the first linked list.

### L3 paramsum

Show how many arguments the program received.

#### WHAT THE SUBJECT ASKS FOR

Show the number of arguments passed to the program, followed by a newline. With no arguments, show `0` and a newline.

Allowed functions: `write`.

```
$> ./paramsum 1 2 3 5 7 24 | cat -e
6$
$> ./paramsum | cat -e
0$
```

#### HOW TO THINK ABOUT IT

- 1 `argc` is already the answer: just subtract 1, which is the program's name.
- 2 There's no `printf`, so you need `ft_putnbr` from the basics.
- 3 It's in level 3 and it's one of the easiest in the whole exam. Don't judge an exercise by its level.

#### THE CODE

paramsum.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 void ft_putnbr(int n)
9 {
10     if (n >= 10)
11         ft_putnbr(n / 10);
12     ft_putchar(n % 10 + '0');
13 }
14
15 int main(int argc, char **argv)
16 {
17     (void)argv;
18     ft_putnbr(argc - 1);
19     ft_putchar('\n');
20     return (0);
21 }
```

#### LINE BY LINE

8-13 The usual recursive `ft_putnbr`.

17 `(void)argv;` is mandatory. `argv` was received and isn't used, and `-Wextra -Werror` turns that warning into a compile error. **Without this line the program doesn't compile.**

18 `argc - 1` removes the program's name from the count.

**WHAT THE SUBJECT ASKS FOR**

Receive a positive number (or zero) in base 10 as text, and show it in base 16 with lowercase letters, followed by a newline.

If the number of parameters is not 1, show only the newline.

Allowed functions: `write`.

```
$> ./print_hex "255" | cat -e
ff$
$> ./print_hex "5156454" | cat -e
4eae66$
```

**HOW TO THINK ABOUT IT**

- ❶ Two independent halves: turn the **text** into a number, then turn that number into **text** in another base.
- ❷ The first half is the times-ten recipe, same as `ft_atoi`.
- ❸ The second is `ft_putnbr` with 16 instead of 10.
- ❹ The problem with writing in base 16 is that the remainders go from 0 to 15, and from 10 up they're no longer digits. The solution avoids any `if`: keep the string `"0123456789abcdef"` and use the remainder as a **position** inside it.

## THE CODE

print\_hex.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  void    put_hex(int n)
9  {
10     char    *base;
11
12     base = "0123456789abcdef";
13     if (n >= 16)
14         put_hex(n / 16);
15     ft_putchar(base[n % 16]);
16 }
17
18 int     main(int argc, char **argv)
19 {
20     int n;
21     int i;
22
23     if (argc == 2)
24     {
25         n = 0;
26         i = 0;
27         while (argv[1][i] >= '0' && argv[1][i] <= '9')
28         {
29             n = n * 10 + (argv[1][i] - '0');
30             i++;
31         }
32         put_hex(n);
33     }
34     ft_putchar('\n');
35     return (0);
36 }
```

### LINE BY LINE

- 12 The digit table. Position 10 is 'a', position 15 is 'f'.
- 13-14 The recursion: if the number still has more than one digit in base 16, the upper part is handled first.
- 15 `n % 16` gives a number between 0 and 15, which is exactly the right position inside the table.
- 27-31 The other half: text to number, with the times-ten recipe.
- 34 The newline outside the `if`, as always.

## L3 ft\_atoi\_base

Function that converts text in any base up to 16 into an integer.

### WHAT THE SUBJECT ASKS FOR

Write a function that converts the received string (in a base up to 16) into a base-10 integer.

The recognized characters are `0123456789abcdef`, cut off depending on the requested base: base 4 recognizes `0123`, base 16 recognizes all of them. Uppercase also counts. The `-` sign is only interpreted if it's the first character.

The declaration is `int ft_atoi_base(const char *str, int str_base);`

## HOW TO THINK ABOUT IT

- 1 It's `ft_atoi` with the base as a variable instead of a fixed 10.
- 2 What makes it harder is knowing when to stop: in base 4, a `7` is already invalid, and in base 16 a `g` is too.
- 3 The solution is to put that question in a standalone function that returns the character's value **or `-1` if it doesn't belong to this base**. That keeps the main loop clean, and it stops by itself when `-1` shows up.

## THE CODE

`ft_atoi_base.c`

```
1  int    value_of(char c, int base)
2  {
3      int v;
4
5      v = -1;
6      if (c >= '0' && c <= '9')
7          v = c - '0';
8      else if (c >= 'a' && c <= 'f')
9          v = c - 'a' + 10;
10     else if (c >= 'A' && c <= 'F')
11         v = c - 'A' + 10;
12     if (v >= base)
13         return (-1);
14     return (v);
15 }
16
17 int    ft_atoi_base(const char *str, int str_base)
18 {
19     int i;
20     int sign;
21     int res;
22     int v;
23
24     i = 0;
25     sign = 1;
26     res = 0;
27     if (str[0] == '-')
28     {
29         sign = -1;
30         i = 1;
31     }
32     v = value_of(str[i], str_base);
33     while (v >= 0)
34     {
35         res = res * str_base + v;
36         i++;
37         v = value_of(str[i], str_base);
38     }
39     return (res * sign);
40 }
```

## LINE BY LINE

- 5 Starts by assuming the character is invalid. If none of the following `if` statements recognize it, it stays that way.
- 6-7 Digits are worth what they look like.
- 8-9 Lowercase `a` to `f` are worth 10 to 15. Hence the `+ 10`.
- 10-11 Uppercase letters are worth the same.
- 12-13 The part that cuts off by base: a `7` is a valid character in general, but not in base 4.
- 27-31 The sign, only if it's really the first character, as the subject requires.
- 32 The first value is read **before** the loop.
- 33 The loop runs while the character is valid in this base.
- 35 The same recipe as `ft_atoi`, with `str_base` instead of 10.
- 37 Reads the next value at the end of each pass, so the `while` condition can be tested.

## L3 tab\_mult

Write the multiplication table of a number.

### WHAT THE SUBJECT ASKS FOR

Show the multiplication table of the received number, from 1 to 9, in the format `1 x 9 = 9`.

If there are no parameters, show only a newline.

Allowed functions: `write`.

```
$> ./tab_mult 9
1 x 9 = 9
2 x 9 = 18
3 x 9 = 27
```

### HOW TO THINK ABOUT IT

- 1 There's no new idea here: it's the times-ten recipe at the start and nine passes writing three numbers and two pieces of text.
- 2 The only trap is the **format**. The spaces around the `x` and the `=` are part of the output and have to be exactly as in the subject.

## THE CODE

tab\_mult.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  void    ft_putnbr(int n)
9  {
10     if (n >= 10)
11         ft_putnbr(n / 10);
12     ft_putchar(n % 10 + '0');
13 }
14
15 int     main(int argc, char **argv)
16 {
17     int n;
18     int i;
19
20     if (argc != 2)
21     {
22         ft_putchar('\n');
23         return (0);
24     }
25     n = 0;
26     i = 0;
27     while (argv[1][i] >= '0' && argv[1][i] <= '9')
28     {
29         n = n * 10 + (argv[1][i] - '0');
30         i++;
31     }
32     i = 1;
33     while (i <= 9)
34     {
35         ft_putnbr(i);
36         write(1, " x ", 3);
37         ft_putnbr(n);
38         write(1, " = ", 3);
39         ft_putnbr(i * n);
40         ft_putchar('\n');
41         i++;
42     }
43     return (0);
44 }
```

## LINE BY LINE

- 27–31** Text to number, same as in `print_hex`.
- 32–33** Nine passes, from 1 to 9.
- 35** This pass's multiplier.
- 36** `" x "` with a space on each side — three bytes. Copy this from the subject instead of typing it from memory.
- 39** The result, computed on the spot.
- 40** The newline at the end of each line of the table.

Always check with `| cat -e`: one space too many or too few between the `x` and the numbers fails the exercise, and it doesn't show at a glance.

**WHAT THE SUBJECT ASKS FOR**

Receive two strictly positive numbers as text and show their greatest common divisor, followed by a newline.

If the number of parameters is not 2, show only the newline.

Allowed functions: `printf`, `atoi`, `malloc`, `free`.

```
$> ./pgcd 42 12 | cat -e
6$
$> ./pgcd 17 3 | cat -e
1$
```

**HOW TO THINK ABOUT IT**

- 1 Once again: **look at the allowed functions first**. With `printf` and `atoi`, the exercise comes down to the algorithm.
- 2 The algorithm is Euclid's, and it's worth knowing by heart, because it also solves `lcm`: replace `(a, b)` with `(b, a mod b)` until `b` is zero. Whatever's left in `a` is the answer.
- 3 Don't try counting down from `a` to 1 by trial — it's more code, slower, and easier to get wrong.

**THE CODE**

pgcd.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char **argv)
5 {
6     int a;
7     int b;
8     int t;
9
10    if (argc != 3)
11    {
12        printf("\n");
13        return (0);
14    }
15    a = atoi(argv[1]);
16    b = atoi(argv[2]);
17    while (b != 0)
18    {
19        t = a % b;
20        a = b;
21        b = t;
22    }
23    printf("%d\n", a);
24    return (0);
25 }
```

**LINE BY LINE**

- |       |                                                                                                          |
|-------|----------------------------------------------------------------------------------------------------------|
| 17    | The loop runs until <code>b</code> is zero.                                                              |
| <hr/> |                                                                                                          |
| 19    | The remainder of the division. It's the algorithm's only calculation.                                    |
| <hr/> |                                                                                                          |
| 20-21 | The two values shift: <code>b</code> becomes <code>a</code> , and the remainder becomes <code>b</code> . |
| <hr/> |                                                                                                          |
| 23    | When <code>b</code> reaches zero, <code>a</code> is the greatest common divisor.                         |

Three lines inside the loop, and it works for any two numbers. It's one of the rare spots in the exam where memorizing the recipe really pays off.

### L3 lcm

Function that returns the least common multiple.

#### WHAT THE SUBJECT ASKS FOR

Write a function that receives two `unsigned int`s and returns their least common multiple. If either one is zero, it returns 0.

The declaration is `unsigned int lcm(unsigned int a, unsigned int b);`

#### HOW TO THINK ABOUT IT

- 1 The subject gives the formula: the least common multiple is the product of the two divided by their greatest common divisor.
- 2 In other words, it's `pgcd` inside, plus one division.
- 3 **The order of the operations matters.** Multiplying first and dividing after can overflow the `unsigned int` limit with large numbers. Dividing first never overflows, because the division's result is always exact.

#### THE CODE

`lcm.c`

```
1 unsigned int  lcm(unsigned int a, unsigned int b)
2 {
3     unsigned int  x;
4     unsigned int  y;
5     unsigned int  t;
6
7     if (a == 0 || b == 0)
8         return (0);
9     x = a;
10    y = b;
11    while (y != 0)
12    {
13        t = x % y;
14        x = y;
15        y = t;
16    }
17    return (a / x * b);
18 }
```

#### LINE BY LINE

7-8 The zero case, which the subject says to handle separately.

9-10 The values are copied, because Euclid's algorithm destroys them and they're still needed at the end.

11-16 Euclid's algorithm, exactly as in `pgcd`. At the end, `x` is the greatest common divisor.

17 `a / x * b`, not `a * b / x`. The two give the same result mathematically, but only the first is safe on a computer.

### L3 add\_prime\_sum

Add up all the primes up to the given number.

#### WHAT THE SUBJECT ASKS FOR

Receive a positive integer and show the sum of every prime number less than or equal to it, followed by a newline.

If the number of arguments is not 1, or if the argument isn't a positive number, show `0` and a newline.

Allowed functions: `write`, `exit`.

```
$> ./add_prime_sum 5 | cat -e
10$
$> ./add_prime_sum 7 | cat -e
17$
$> ./add_prime_sum | cat -e
0$
```

#### HOW TO THINK ABOUT IT

- ❶ Two parts: knowing whether a number is prime, and summing the ones that are.
- ❷ To know if it's prime, try dividing by 2, 3, 4... But you don't need to go all the way: **it's enough to go up to the square root**. If a number has a divisor larger than its square root, it must have another one smaller, which would already have been found. Written as `i * i <= n`, which avoids computing square roots.
- ❸ Validating the argument is half the exercise: the subject asks for `0` both when there's no argument and when it isn't a number.

## THE CODE

add\_prime\_sum.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  void    ft_putnbr(int n)
9  {
10     if (n >= 10)
11         ft_putnbr(n / 10);
12     ft_putchar(n % 10 + '0');
13 }
14
15 int     is_prime(int n)
16 {
17     int i;
18
19     if (n < 2)
20         return (0);
21     i = 2;
22     while (i * i <= n)
23     {
24         if (n % i == 0)
25             return (0);
26         i++;
27     }
28     return (1);
29 }
30
31 int     main(int argc, char **argv)
32 {
33     int n;
34     int i;
35     int sum;
36
37     i = 0;
38     if (argc == 2)
39         while (argv[1][i] >= '0' && argv[1][i] <= '9')
40             i++;
41     if (argc != 2 || i == 0 || argv[1][i] != '\0')
42     {
43         write(1, "0\n", 2);
44         return (0);
45     }
46     n = 0;
47     i = 0;
48     while (argv[1][i])
49         n = n * 10 + (argv[1][i++] - '0');
50     sum = 0;
51     i = 2;
52     while (i <= n)
53     {
54         if (is_prime(i))
55             sum = sum + i;
56         i++;
57     }
58     ft_putnbr(sum);
59     ft_putchar('\n');
60     return (0);
61 }
```

## LINE BY LINE

19–20 Neither 0 nor 1 is prime, and negatives aren't either.

22 `i * i <= n` is the same as "`i` up to the square root of `n`", without computing any roots.

24–25 Found a divisor, so it's not prime, and it exits right away.

28 If no divisor was found, it's prime.

37–40 Counts how many digits are at the start of the argument.

41 Three error cases in one line: not exactly one argument; doesn't start with a digit; or there's something after the digits.

48–49 Text to number, with the `i++` inside the brackets.

52–57 Walks every number from 2 to `n` and adds up the ones that are prime.

## L3 epur\_str

The string with exactly one space between words.

### WHAT THE SUBJECT ASKS FOR

Show the string with exactly one space between words, no spaces or tabs at the start or end, followed by a newline.

If the number of arguments is not 1, or if there are no words, show only the newline.

Allowed functions: `write`.

```
$> ./epur_str " seulement la c'est plus dur " | cat -e
seulement la c'est plus dur$
```

### HOW TO THINK ABOUT IT

- 1 It's `first_word` put inside a loop that repeats: skip separators, write the word, again.
- 2 The detail that decides the exercise: **the separating space is written before each word, except the first one**. If it were written after each word, there'd be one extra space at the end and the output would fail — and that's the number-one mistake in this exercise.
- 3 To know whether a word has already been written, use a flag.

## THE CODE

epur\_str.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     is_space(char c)
9  {
10     return (c == ' ' || c == '\t');
11 }
12
13 int     main(int argc, char **argv)
14 {
15     int i;
16     int first;
17
18     if (argc == 2)
19     {
20         i = 0;
21         first = 1;
22         while (argv[1][i])
23         {
24             while (is_space(argv[1][i]))
25                 i++;
26             if (argv[1][i] && !first)
27                 ft_putchar(' ');
28             while (argv[1][i] && !is_space(argv[1][i]))
29             {
30                 ft_putchar(argv[1][i]);
31                 i++;
32                 first = 0;
33             }
34         }
35     }
36     ft_putchar('\n');
37     return (0);
38 }
```

### LINE BY LINE

|       |                                                                                                                                                                                                    |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 21    | The flag starts saying "I haven't written anything yet".                                                                                                                                           |
| 22    | The outer loop repeats the skip/write pair until the end of the string.                                                                                                                            |
| 24-25 | Skips every separator in a row.                                                                                                                                                                    |
| 26    | Writes the separating space <b>before</b> the word, and only if there was already another one before it.                                                                                           |
| 28-33 | Writes the word character by character.                                                                                                                                                            |
| 32    | As soon as a character comes out, the flag switches to "I've already written something". It's inside the loop on purpose: a string with only spaces never reaches here and the flag never changes. |

## L3 expand\_str

The same, but with three spaces between words.

### WHAT THE SUBJECT ASKS FOR

Exactly the same as `epur_str`, but with three spaces between words instead of one.

Allowed functions: `write`.

```
$> ./expand_str " seulement la c'est plus dur " | cat -e
seulement la c'est plus dur$
```

## HOW TO THINK ABOUT IT

- 1 One line changes. Everything else is identical to `epur_str`.

## THE CODE

expand\_str.c

```
1 #include <unistd.h>
2
3 void ft_putchar(char c)
4 {
5     write(1, &c, 1);
6 }
7
8 int is_space(char c)
9 {
10     return (c == ' ' || c == '\t');
11 }
12
13 int main(int argc, char **argv)
14 {
15     int i;
16     int first;
17
18     if (argc == 2)
19     {
20         i = 0;
21         first = 1;
22         while (argv[1][i])
23         {
24             while (is_space(argv[1][i]))
25                 i++;
26             if (argv[1][i] && !first)
27                 write(1, "   ", 3);
28             while (argv[1][i] && !is_space(argv[1][i]))
29             {
30                 ft_putchar(argv[1][i]);
31                 i++;
32                 first = 0;
33             }
34         }
35     }
36     ft_putchar('\n');
37     return (0);
38 }
```

## LINE BY LINE

- 27 Used to be `ft_putchar(' ')`; now it writes three spaces at once. The `3` is how many bytes are written.

## L3 str\_capitalizer

The first letter of each word in uppercase, the rest lowercase.

### WHAT THE SUBJECT ASKS FOR

For **each argument**, put the first letter of each word in uppercase and the rest in lowercase, and show the result followed by a newline. A one-letter word stays uppercase.

If there are no arguments, show only a newline.

Allowed functions: `write`.

```
$> ./str_capitalizer "Premier PETIT Test" | cat -e
Premier Petit Test$
```

## HOW TO THINK ABOUT IT

- 1 Two new ideas. The first: this exercise accepts **several** arguments, so there's an outer loop passing through all of them.
- 2 The second: how do you know you're at the start of a word? By keeping a flag that answers "was the previous character a separator?". The flag starts at 1, because the first character of the string is always the start of a word.
- 3 The organizing trick: **lowercase everything first, and only then raise the initial**. Doing the two things in order is much simpler than trying to decide everything at once.

## THE CODE

### str\_capitalizer.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     is_space(char c)
9  {
10     return (c == ' ' || c == '\t');
11 }
12
13 int     main(int argc, char **argv)
14 {
15     int    a;
16     int    i;
17     int    start;
18     char    c;
19
20     a = 1;
21     while (a < argc)
22     {
23         i = 0;
24         start = 1;
25         while (argv[a][i])
26         {
27             c = argv[a][i];
28             if (c >= 'A' && c <= 'Z')
29                 c = c + 32;
30             if (start && c >= 'a' && c <= 'z')
31                 c = c - 32;
32             start = is_space(argv[a][i]);
33             ft_putchar(c);
34             i++;
35         }
36         ft_putchar('\n');
37         a++;
38     }
39     if (argc == 1)
40         ft_putchar('\n');
41     return (0);
42 }
```

## LINE BY LINE

|       |                                                                                                                                                                                                                            |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 21    | The outer loop: <code>a</code> goes from 1 to the last argument.                                                                                                                                                           |
| 24    | The flag starts saying "I'm at the start of a word".                                                                                                                                                                       |
| 28-29 | Step 1: lowercase everything.                                                                                                                                                                                              |
| 30-31 | Step 2: if it's the start of a word and it's a letter, raise it to uppercase.                                                                                                                                              |
| 32    | Updates the flag for the <b>next</b> character: it now means "was what I just saw a separator?". Notice it looks at <code>argv[a][i]</code> , the original character, not <code>c</code> , which has already been changed. |
| 36    | One newline per argument.                                                                                                                                                                                                  |
| 39-40 | The case of no arguments: the outer loop never runs, so the newline has to be written here.                                                                                                                                |

## L3 rstr\_capitalizer

The last letter of each word in uppercase, the rest lowercase.

### WHAT THE SUBJECT ASKS FOR

The opposite of `str_capitalizer`: for each argument, put the **last** letter of each word in uppercase and the rest lowercase. A one-letter word stays uppercase.

If there are no arguments, show only a newline.

Allowed functions: `write`.

```
$> ./rstr_capitalizer "Premier PETIT TeST" | cat -e
premieR petiT test$
```

### HOW TO THINK ABOUT IT

- 1 No flag is needed here. "Am I at the end of a word?" is a question about the **next** character, and you can already see that: it's `argv[a][i + 1]`.
- 2 It's safe to peek ahead: if character `i` exists, then `i + 1` is at worst the `'\0'`, which is also there.
- 3 End of word means: the next one is the `'\0'` or a separator.

## THE CODE

rstr\_capitalizer.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     is_space(char c)
9  {
10     return (c == ' ' || c == '\t');
11 }
12
13 int     main(int argc, char **argv)
14 {
15     int    a;
16     int    i;
17     char   c;
18
19     a = 1;
20     while (a < argc)
21     {
22         i = 0;
23         while (argv[a][i])
24         {
25             c = argv[a][i];
26             if (c >= 'A' && c <= 'Z')
27                 c = c + 32;
28             if (c >= 'a' && c <= 'z'
29                 && (argv[a][i + 1] == '\0' || is_space(argv[a][i + 1])))
30                 c = c - 32;
31             ft_putchar(c);
32             i++;
33         }
34         ft_putchar('\n');
35         a++;
36     }
37     if (argc == 1)
38         ft_putchar('\n');
39     return (0);
40 }
```

## LINE BY LINE

26-27 Step 1, same as the previous exercise: lowercase everything.

28-29 Step 2: it's a letter **and** the following character is the end of the string or a separator.

30 Only then is it raised to uppercase.

## L3 hiddenp

Say whether the first string is hidden inside the second.

### WHAT THE SUBJECT ASKS FOR

Show `1` followed by a newline if the first string is hidden inside the second, otherwise `0` followed by a newline.

`s1` is hidden in `s2` if every character of `s1` can be found inside `s2`, in the same order. The empty string is hidden in any string.

If the number of parameters is not 2, show only a newline.

Allowed functions: `write`.

```
$> ./hiddenp "abc" "2altrb53c.sse" | cat -e
1$
$> ./hiddenp "abc" "btarc" | cat -e
0$
```

## HOW TO THINK ABOUT IT

- 1 It's exactly the same algorithm as `wdmatch`, from level 2. The only difference is what gets written at the end.
- 2 One index in each string. The second one's always moves; the first one's only advances when it finds the character it's looking for.
- 3 If at the end the first string's index has reached `'\0'`, everything was found in the right order.

## THE CODE

hiddenp.c

```
1 #include <unistd.h>
2
3 int main(int argc, char **argv)
4 {
5     int i;
6     int j;
7
8     if (argc != 3)
9     {
10         write(1, "\n", 1);
11         return (0);
12     }
13     i = 0;
14     j = 0;
15     while (argv[1][i] && argv[2][j])
16     {
17         if (argv[1][i] == argv[2][j])
18             i++;
19         j++;
20     }
21     if (!argv[1][i])
22         write(1, "1\n", 2);
23     else
24         write(1, "0\n", 2);
25     return (0);
26 }
```

## LINE BY LINE

- |       |                                                                                                       |
|-------|-------------------------------------------------------------------------------------------------------|
| 15    | Walks while there are characters in both strings.                                                     |
| 17-18 | The first string's index only advances on a match.                                                    |
| 19    | The second one's always advances.                                                                     |
| 21    | <code>!argv[1][i]</code> means "I've reached the end of the first string", i.e. everything was found. |

If you solve `wdmatch`, do this one next — they're the same exercise across two levels, and doing both learns them at once.

## L3 ft\_range

Function that creates an array with every integer between two values.

### WHAT THE SUBJECT ASKS FOR

Write the function `int *ft_range(int start, int end);`

It has to allocate with `malloc` an array of integers, fill it with the consecutive values from `start` to `end` (including both), and return the pointer to the first value.

With `(1, 3)` it returns 1, 2, 3. With `(0, -3)` it returns 0, -1, -2, -3.

Allowed functions: `malloc`.

### HOW TO THINK ABOUT IT

- 1 The three `malloc` steps again: count, request, fill in.
- 2 There's only one difficulty: **the array can go up or down**, and the subject gives examples of both.
- 3 It's solved with a step (`step`) worth 1 or -1, decided **once** at the start. After that, the math works in both cases without repeating code.
- 4 `(end - start) * step` is always positive, whichever direction it goes, because `step` corrects the sign. Plus one, because both ends count.

### THE CODE

ft\_range.c

```
1 #include <stdlib.h>
2
3 int *ft_range(int start, int end)
4 {
5     int *tab;
6     int len;
7     int step;
8     int i;
9
10    step = 1;
11    if (start > end)
12        step = -1;
13    len = (end - start) * step + 1;
14    tab = malloc(sizeof(int) * len);
15    if (!tab)
16        return (0);
17    i = 0;
18    while (i < len)
19    {
20        tab[i] = start + i * step;
21        i++;
22    }
23    return (tab);
24 }
```

### LINE BY LINE

10-12 Decides the direction, once.

13 How many elements. Multiplying by `step` makes the difference positive in both cases.

14 Requests the space. `sizeof(int)` because these are integers, not characters.

15-16 Checking `malloc` is mandatory. Without it, you lose points even if the program seems to work.

20 The value of each position: starts at `start` and walks `i` steps. With `step = -1`, it walks backwards on its own.

Without the `step` idea, this exercise would need everything written twice inside an `if`. It's the same kind of simplification as `'a' + i` in `maff_alpha`.

### L3 ft\_rrange

The same array, in reverse order.

#### WHAT THE SUBJECT ASKS FOR

Exactly the same as `ft_range`, but with the values in reverse order: it starts at `end` and finishes at `start`.

With `(1, 3)` it returns 3, 2, 1. With `(0, -3)` it returns -3, -2, -1, 0.

Allowed functions: `malloc`.

#### HOW TO THINK ABOUT IT

❶ One line changes: instead of starting from `start` and going forward, you start from `end` and go backward.

#### THE CODE

ft\_rrange.c

```
1 #include <stdlib.h>
2
3 int *ft_rrange(int start, int end)
4 {
5     int *tab;
6     int len;
7     int step;
8     int i;
9
10    step = 1;
11    if (start > end)
12        step = -1;
13    len = (end - start) * step + 1;
14    tab = malloc(sizeof(int) * len);
15    if (!tab)
16        return (0);
17    i = 0;
18    while (i < len)
19    {
20        tab[i] = end - i * step;
21        i++;
22    }
23    return (tab);
24 }
```

#### LINE BY LINE

20 Used to be `start + i * step`; it becomes `end - i * step`. Check it by hand with `(0, -3)`: `step` is -1, and `-3 - i * (-1)` gives -3, -2, -1, 0. Correct.

### L3 ft\_list\_size

Function that counts the elements of a linked list.

#### WHAT THE SUBJECT ASKS FOR

Write a function that returns the number of elements in the list it receives.

The declaration is `int ft_list_size(t_list *begin_list);`

You turn in two files, and `ft_list.h` has to have exactly the structure from the subject.

## HOW TO THINK ABOUT IT

- 1 A linked list is a chain: each link holds a piece of data and the address of the next link. The last one points to `NULL`, which is the end marker — the equivalent of `'\0'` for strings.
- 2 So the loop is the sibling of the string loop: `while (list)` instead of `while (s[i])`.
- 3 Moving to the next link is `list = list->next`.
- 4 You can freely modify the pointer you received: it's a copy of the address, and the caller's list stays intact.

## THE CODE

ft\_list\_size.c

```
1 #include "ft_list.h"
2
3 int ft_list_size(t_list *begin_list)
4 {
5     int n;
6
7     n = 0;
8     while (begin_list)
9     {
10         n++;
11         begin_list = begin_list->next;
12     }
13     return (n);
14 }
```

ft\_list.h

```
1 #ifndef FT_LIST_H
2 # define FT_LIST_H
3
4 typedef struct s_list
5 {
6     struct s_list *next;
7     void *data;
8 } t_list;
9
10 #endif
```

## LINE BY LINE

- 8 `while (begin_list)` reads as "while the pointer isn't `NULL`". `NULL` is worth zero, so it counts as false, just like `'\0'`.
- 
- 10 One more link counted.
- 
- 11 `->` is how you reach a field through a pointer. `begin_list->next` is the address of the next link.

The header's `#ifndef` is the standard guard for `.h` files: it prevents the file from being included twice. It's not mandatory, but it never hurts. The `typedef` has to be copied **from the subject**, not written from memory.

## L4 Level 4

11 exercises. Memory inside memory, sorting, and pointers to functions. This is where most people get stuck.

### L4 sort\_int\_tab

Function that sorts an array of integers in place.

#### WHAT THE SUBJECT ASKS FOR

Write the function `void sort_int_tab(int *tab, unsigned int size);`

Sorts the array in ascending order, in place. Duplicate values all stay.

#### HOW TO THINK ABOUT IT

- 1 This is the easiest in level 4 — start with this one.
- 2 The simplest idea that works: walk the array comparing each element with the next one. If they're out of order, swap them. Then go back to the start. When you reach the end without swapping anything, it's sorted.
- 3 Going back to the start on every swap looks slow, and it is, but the exam doesn't measure speed — it measures whether the output is right.
- 4 The swap is the same as `ft_swap`: it needs a temporary variable.

#### THE CODE

sort\_int\_tab.c

```
1 void sort_int_tab(int *tab, unsigned int size)
2 {
3     unsigned int i;
4     int tmp;
5
6     i = 0;
7     while (i + 1 < size)
8     {
9         if (tab[i] > tab[i + 1])
10        {
11            tmp = tab[i];
12            tab[i] = tab[i + 1];
13            tab[i + 1] = tmp;
14            i = 0;
15        }
16        else
17            i++;
18    }
19 }
```

#### LINE BY LINE

- |       |                                                                                                                                                                               |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3     | <code>i</code> has to be <code>unsigned int</code> like <code>size</code> . Comparing types with different signedness triggers a warning, and a warning is a failed exercise. |
| 7     | <code>i + 1 &lt; size</code> guarantees that <code>tab[i + 1]</code> on the next line always exists.                                                                          |
| 9     | Are they out of order?                                                                                                                                                        |
| 11-13 | The three-step swap, with <code>tmp</code> holding one of the values.                                                                                                         |
| 14    | <b>Back to the start.</b> This line is what makes the algorithm work: a swap might have disturbed something further back.                                                     |
| 16-17 | If they were already in order, move on.                                                                                                                                       |

**WHAT THE SUBJECT ASKS FOR**

Receive a positive integer and show its prime factors in increasing order, separated by `*`, followed by a newline.

If the number of parameters is not 1, show only the newline. The input number, when there is one, is always valid.

Allowed functions: `printf`, `atoi`.

```
$> ./fprime 225225 | cat -e
3*3*5*5*7*11*13$
$> ./fprime 42 | cat -e
2*3*7$
$> ./fprime 1 | cat -e
1$
```

**HOW TO THINK ABOUT IT**

- ❶ You don't need to know which numbers are prime. Just divide by 2 as long as it works, then by 3 as long as it works, then by 4... By the time you reach 4, the number no longer has any factors of 2 left, so the division never succeeds. The divisors that do work are automatically prime.
- ❷ `1` is a special case: it has no factors, and the subject says the output is `1`.
- ❸ The `*` is written **before** each factor except the first — the same idea as the separator in `epur_str`.

## THE CODE

fprime.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int  main(int argc, char **argv)
5  {
6      int n;
7      int d;
8      int first;
9
10     if (argc != 2)
11     {
12         printf("\n");
13         return (0);
14     }
15     n = atoi(argv[1]);
16     if (n <= 1)
17     {
18         printf("%d\n", n);
19         return (0);
20     }
21     first = 1;
22     d = 2;
23     while (n > 1)
24     {
25         while (n % d == 0)
26         {
27             if (!first)
28                 printf("*");
29             printf("%d", d);
30             first = 0;
31             n = n / d;
32         }
33         d++;
34     }
35     printf("\n");
36     return (0);
37 }
```

### LINE BY LINE

- 16-20 The case of 1 (and 0), handled separately.
- 21 The separator flag.
- 23 While there's still something left to break down.
- 25 While the current divisor still fits, it's applied repeatedly — that's how 225225 produces two 3s and two 5s.
- 27-28 The `*` before every factor except the first.
- 30 From here on a factor has already been written, so the following ones get a `*`.
- 31 The factor is removed from the number.
- 33 Only once the divisor no longer fits do you move to the next one.

## L4 rostring

Move the first word to the end of the sentence.

### WHAT THE SUBJECT ASKS FOR

Show the string after rotating it one word to the left: the first word becomes the last, and the others keep their order. The words come out separated by a single space.

If there is less than one argument, show only a newline.

Allowed functions: `write`, `malloc`, `free`.

```
$> ./rostring "Que la      lumiere soit et la lumiere fut"
la lumiere soit et la lumiere fut Que
$> ./rostring "abc  " | cat -e
abc$
```

#### HOW TO THINK ABOUT IT

- ❶ No `malloc` is actually needed, even though it's allowed: everything can be written directly in the right order.
- ❷ The plan: remember where the first word starts and ends, write every other word with `epur_str`-style spacing, and write the first word last.
- ❸ The second example is the case that catches a lot of people: if there's only one word, no space can come out before it.

## THE CODE

rostring.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     is_space(char c)
9  {
10     return (c == ' ' || c == '\t');
11 }
12
13 int     main(int argc, char **argv)
14 {
15     int i;
16     int start;
17     int end;
18     int written;
19
20     if (argc < 2)
21     {
22         ft_putchar('\n');
23         return (0);
24     }
25     i = 0;
26     while (is_space(argv[1][i]))
27         i++;
28     start = i;
29     while (argv[1][i] && !is_space(argv[1][i]))
30         i++;
31     end = i;
32     written = 0;
33     while (argv[1][i])
34     {
35         while (is_space(argv[1][i]))
36             i++;
37         if (argv[1][i] && written)
38             ft_putchar(' ');
39         while (argv[1][i] && !is_space(argv[1][i]))
40         {
41             ft_putchar(argv[1][i]);
42             written = 1;
43             i++;
44         }
45     }
46     if (written && end > start)
47         ft_putchar(' ');
48     while (start < end)
49         ft_putchar(argv[1][start++]);
50     ft_putchar('\n');
51     return (0);
52 }
```

## LINE BY LINE

- 25–31 Locates the first word: skips the leading separators, marks `start`, walks past the word, marks `end`. Nothing is written yet.
- 
- 32 The "have I written a word yet" flag, same as in `epur_str`.
- 
- 33–45 Writes every word from the second one onward, with a space before each except the first one written.
- 
- 46–47 The space before the first word, which now goes at the end. It's only written if something has already come out **and** the first word actually exists.
- 
- 48–49 Writes the first word, from `start` to `end`. The `start++` inside the brackets advances the index at the same time it's used.
- 

## L4 rev\_wstr

Write the words in reverse order.

### WHAT THE SUBJECT ASKS FOR

Show the words of the string in reverse order, followed by a newline. It's the order of the **words** that gets reversed, not the letters of each one.

If the number of parameters is not 1, show only the newline.

Allowed functions: `write`, `malloc`, `free`.

```
$> ./rev_wstr "il contempla le mont" | cat -e
mont le contempla il$
```

### HOW TO THINK ABOUT IT

- 1 It's `last_word` put inside a loop: find the last word, write it, and keep stepping back to the previous one.
- 2 It doesn't need `malloc` either: walk the string backwards and write as you go.
- 3 Each pass: skip the separators going backward, mark the end of the word, step back over the word, and write it from its start to its end.
- 4 Notice that after stepping back over the word, the index sits **right before** it, which is exactly where the next pass needs to continue — nothing needs correcting.

## THE CODE

rev\_wstr.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  int     is_space(char c)
9  {
10     return (c == ' ' || c == '\t');
11 }
12
13 int     main(int argc, char **argv)
14 {
15     int i;
16     int j;
17     int end;
18     int written;
19
20     if (argc != 2)
21     {
22         ft_putchar('\n');
23         return (0);
24     }
25     i = 0;
26     while (argv[1][i])
27         i++;
28     i--;
29     written = 0;
30     while (i >= 0)
31     {
32         while (i >= 0 && is_space(argv[1][i]))
33             i--;
34         end = i;
35         while (i >= 0 && !is_space(argv[1][i]))
36             i--;
37         if (end > i)
38         {
39             if (written)
40                 ft_putchar(' ');
41             written = 1;
42             j = i + 1;
43             while (j <= end)
44                 ft_putchar(argv[1][j++]);
45         }
46     }
47     ft_putchar('\n');
48     return (0);
49 }
```

## LINE BY LINE

|       |                                                                                                                                                 |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 26–28 | Goes to the end of the string and steps back one, landing on the last real character.                                                           |
| 30    | The outer loop walks backward to the start of the string.                                                                                       |
| 32–33 | Skips the separators, going backward.                                                                                                           |
| 34    | Marks the end of the word.                                                                                                                      |
| 35–36 | Steps back over the entire word.                                                                                                                |
| 37    | <code>end &gt; i</code> means "I really found a word". If the string ends in separators, this condition keeps an empty word from being written. |
| 39–40 | The space before each word except the first one to come out.                                                                                    |
| 42–44 | Writes the word forward: starts at <code>i + 1</code> , its first character, and goes up to <code>end</code> .                                  |

## L4 ft\_split

Function that splits a string into an array of words.

### WHAT THE SUBJECT ASKS FOR

Write a function that splits a string into words and returns them as an array of strings ending in `NULL`.

A word is a chunk of the string delimited by spaces, tabs or newlines, or by the start and end of the string.

The declaration is `char **ft_split(char *str);`

Allowed functions: `malloc`.

### HOW TO THINK ABOUT IT

- 1 There's `malloc` inside `malloc` here: one for the **list of pointers** and one per word.
- 2 So you need to know how many words there are **before** requesting the list — hence the function that only counts.
- 3 The `NULL` at the end is what lets whoever calls it know where the list stops, just like `'\0'` for strings. Forgetting it is the classic mistake in this exercise.
- 4 Splitting the work into three small functions (count, copy one word, assemble) makes this much easier than trying to do it all in one.

## THE CODE

`ft_split.c`

```

1  #include <stdlib.h>
2
3  int    is_sep(char c)
4  {
5      return (c == ' ' || c == '\t' || c == '\n');
6  }
7
8  int    count_words(char *str)
9  {
10     int i;
11     int n;
12
13     i = 0;
14     n = 0;
15     while (str[i])
16     {
17         while (str[i] && is_sep(str[i]))
18             i++;
19         if (str[i])
20             n++;
21         while (str[i] && !is_sep(str[i]))
22             i++;
23     }
24     return (n);
25 }
26
27 char    *word_dup(char *str, int start, int end)
28 {
29     char    *w;
30     int      i;
31
32     w = malloc(sizeof(char) * (end - start + 1));
33     if (!w)
34         return (0);
35     i = 0;
36     while (start < end)
37         w[i++] = str[start++];
38     w[i] = '\0';
39     return (w);
40 }
41
42 char    **ft_split(char *str)
43 {
44     char    **tab;
45     int      i;
46     int      j;
47     int      start;
48
49     tab = malloc(sizeof(char *) * (count_words(str) + 1));
50     if (!tab)
51         return (0);
52     i = 0;
53     j = 0;
54     while (str[i])
55     {
56         while (str[i] && is_sep(str[i]))
57             i++;
58         start = i;
59         while (str[i] && !is_sep(str[i]))
60             i++;
61         if (i > start)
62             tab[j++] = word_dup(str, start, i);
63     }
64     tab[j] = 0;
65     return (tab);
66 }

```

## LINE BY LINE

|       |                                                                                                                                                                                |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3-6   | The separators. Here the subject includes <code>\n</code> , unlike the word exercises in earlier levels.                                                                       |
| 8-25  | Only counts words. Same pair of loops as always, but instead of writing, it adds one.                                                                                          |
| 19-20 | Counts the word the moment it finds it, before walking past it.                                                                                                                |
| 27-40 | Copies one word into new memory. Receives where it starts and where it ends.                                                                                                   |
| 32    | <code>end - start</code> is the word's length; the <code>+ 1</code> is the space for the <code>'\0'</code> .                                                                   |
| 38    | The <code>'\0'</code> set by hand, as always after a <code>malloc</code> .                                                                                                     |
| 49    | The list of pointers: as many as the words, plus one for the final <code>NULL</code> . <code>sizeof(char *)</code> because these are <b>string addresses</b> , not characters. |
| 61-62 | Only stores the word if it really exists. <code>j++</code> advances the position in the list.                                                                                  |
| 64    | The final <code>NULL</code> . Without it, whoever walks the list doesn't know where to stop.                                                                                   |

## L4 ft\_itoa

Function that converts an integer into a new string.

### WHAT THE SUBJECT ASKS FOR

Write a function that receives an `int` and converts it into a string terminated by zero. The string is returned in memory the function has to allocate.

The declaration is `char *ft_itoa(int nbr);`

Allowed functions: `malloc`.

### HOW TO THINK ABOUT IT

- 1 It's the reverse of `ft_atoi`, and also `ft_putnbr` storing instead of writing.
- 2 Since `malloc` is needed, the size has to be known **beforehand**: count how many digits the number has by dividing by 10 until only one is left.
- 3 If it's negative, that's one more character for the sign.
- 4 It's filled in **back to front**, because the remainder of dividing by 10 always gives the **last** digit. That way nothing needs reversing at the end.
- 5 `n` is declared `unsigned int` rather than `int` because of `-2147483648`: that number has no positive counterpart inside an `int`, and `-n` would give the same number back. Inside an `unsigned int`, 2147483648 fits easily.

## THE CODE

ft\_itoa.c

```
1  #include <stdlib.h>
2
3  char  *ft_itoa(int nbr)
4  {
5      char      *s;
6      unsigned int  n;
7      int      len;
8      int      neg;
9
10     neg = 0;
11     n = nbr;
12     if (nbr < 0)
13     {
14         neg = 1;
15         n = -(unsigned int)nbr;
16     }
17     len = 1;
18     while (n >= 10)
19     {
20         n = n / 10;
21         len++;
22     }
23     len = len + neg;
24     s = malloc(sizeof(char) * (len + 1));
25     if (!s)
26         return (0);
27     s[len] = '\0';
28     n = nbr;
29     if (neg)
30         n = -(unsigned int)nbr;
31     while (len-- > neg)
32     {
33         s[len] = n % 10 + '0';
34         n = n / 10;
35     }
36     if (neg)
37         s[0] = '-';
38     return (s);
39 }
```

## LINE BY LINE

- 6 `unsigned int` instead of `int` — that's what lets you store the opposite of the smallest possible `int`, the 2147483648 that doesn't fit in one. A `long` also works on Linux, but on Windows `long` is 32 bits and it wouldn't fit again; `unsigned int` works on both.
- 12–16 If it's negative, mark it and work with the positive value. `-(unsigned int)nbr` converts **first** and only then flips the sign: for `unsigned`, that operation is circular and defined by the standard, unlike `-nbr` on an `int`, which for `-2147483648` is overflow.
- 17–22 Counts the digits: divides by 10 until only one is left. `len` starts at 1 because even zero has one digit.
- 23 If it's negative, one more character for the `-`.
- 24 Requests the space, with the `+ 1` for the `'\0'`.
- 27 The `'\0'` at the end, written right away.
- 28–30 Restores the number, because the counting loop destroyed it.
- 31 `len-- > neg` does two things: compares and then steps back. It stops at `neg`, leaving position 0 free for the sign when needed.
- 33 `n % 10` is the last digit; adding `'0'` turns it into a character.
- 36–37 The sign, in the first position.

## L4 ft\_list\_foreach

Function that applies a function to every element of a list.

### WHAT THE SUBJECT ASKS FOR

Write a function that receives a list and a function pointer, and applies that function to every element of the list.

The declaration is `void ft_list_foreach(t_list *begin_list, void (*f)(void *));`

The function pointed to by `f` is used like this: `(*f)(list_ptr->data);`

**You turn in two files:** the `.c` and `ft_list.h` with the structure from the subject.

### HOW TO THINK ABOUT IT

- 1 It's the `ft_list_size` loop with one different line inside it.
- 2 The new part is the **function pointer**: `f` isn't a value, it's the address of a function written somewhere else. Calling it is written `(*f)(argument)`.
- 3 You don't need to understand what the function does — just call it with each link's data.

### THE CODE

`ft_list_foreach.c`

```
1 #include "ft_list.h"
2
3 void ft_list_foreach(t_list *begin_list, void (*f)(void *))
4 {
5     while (begin_list)
6     {
7         (*f)(begin_list->data);
8         begin_list = begin_list->next;
9     }
10 }
```

#### ft\_list.h

```
1 #ifndef FT_LIST_H
2 # define FT_LIST_H
3
4 typedef struct s_list
5 {
6     struct s_list *next;
7     void *data;
8 } t_list;
9
10 #endif
```

#### LINE BY LINE

- 3 `void (*f)(void *)` reads from the inside out: `f` is a pointer to a function that receives a `void *` and returns nothing.
- 5 The list loop: walks while the pointer isn't `NULL`.
- 7 Calls the function with this link's data, exactly as the subject says to write it.
- 8 Moves to the next link.

### L4 ft\_list\_remove\_if

Function that removes from the list the elements equal to a reference value.

#### WHAT THE SUBJECT ASKS FOR

Write a function that removes from the list every element whose data is "equal" to a reference value.

The declaration is `void ft_list_remove_if(t_list **begin_list, void *data_ref, int (*cmp)());`

`cmp` receives two `void *` and returns 0 when the two are equal.

Allowed functions: `free`.

#### HOW TO THINK ABOUT IT

- ❶ Notice two things in the subject. `cmp` returns **0 when they're equal** — the opposite of what intuition says. And the first parameter is `t_list **`, i.e. the address **of the pointer** to the start: that's how the function can change the list's head.
- ❷ Removing a link from the middle is easy: the previous link points to the next one, and the middle one is freed.
- ❸ Removing the **head** is different, because there's no previous link. So it's handled first, in its own loop, until the head no longer needs removing.
- ❹ After removing a link from the middle, **you don't move forward**: the next link might also need removing.

## THE CODE

ft\_list\_remove\_if.c

```
1 #include <stdlib.h>
2 #include "ft_list.h"
3
4 void ft_list_remove_if(t_list **begin_list, void *data_ref, int (*cmp)())
5 {
6     t_list *cur;
7     t_list *tmp;
8
9     while (*begin_list && cmp((*begin_list)->data, data_ref) == 0)
10    {
11        tmp = (*begin_list)->next;
12        free(*begin_list);
13        *begin_list = tmp;
14    }
15    cur = *begin_list;
16    while (cur && cur->next)
17    {
18        if (cmp(cur->next->data, data_ref) == 0)
19        {
20            tmp = cur->next->next;
21            free(cur->next);
22            cur->next = tmp;
23        }
24        else
25            cur = cur->next;
26    }
27 }
```

## LINE BY LINE

- |       |                                                                                                                                       |
|-------|---------------------------------------------------------------------------------------------------------------------------------------|
| 9     | First loop: while the <b>head</b> needs removing. <code>*begin_list</code> is the pointer to the first link.                          |
| 11-13 | Saves the next one, frees the head, and the head becomes that next one. This is where the caller's list actually changes.             |
| 15    | From here on the head is guaranteed to be fine.                                                                                       |
| 16    | Second loop: always looks at the link <b>after</b> the current one, because you need the previous one at hand to reconnect the chain. |
| 20-22 | Saves the link after the one about to disappear, frees it, and links the current one directly to that one. The chain stays closed.    |
| 24-25 | Only moves forward when the next link wasn't removed.                                                                                 |

## L4 sort\_list

Function that sorts a linked list with a given comparator.

### WHAT THE SUBJECT ASKS FOR

Write the function `t_list *sort_list(t_list* lst, int (*cmp)(int, int));`

Sorts the list using `cmp` to decide the order, and returns the pointer to the first element. Duplicate values stay.

`cmp` returns a value other than 0 when `a` and `b` **are in the right order**. For example, this function used as `cmp` sorts in ascending order:

```
int ascending(int a, int b) { return (a <= b); }
```

The `t_list` type comes from the `list.h` file that's provided. **You include that file but you don't turn it in.**

### HOW TO THINK ABOUT IT

- 1 It's `sort_int_tab` applied to a list.

- ❷ The trick that simplifies everything: **swap the data instead of the links**. Rearranging pointers is much more complicated, and the result looks exactly the same from the outside.
- ❸ Whenever there's a swap, go back to the start of the list, just like in `sort_int_tab`.
- ❹ Watch the direction of `cmp`: it returns true when things are **fine**, so you swap when it returns false — hence the `!`.

#### THE CODE

`sort_list.c`

```
1  #include "list.h"
2
3  t_list *sort_list(t_list *lst, int (*cmp)(int, int))
4  {
5      t_list *cur;
6      int     tmp;
7
8      cur = lst;
9      while (cur && cur->next)
10     {
11         if (!(*cmp)(cur->data, cur->next->data))
12         {
13             tmp = cur->data;
14             cur->data = cur->next->data;
15             cur->next->data = tmp;
16             cur = lst;
17         }
18         else
19             cur = cur->next;
20     }
21     return (lst);
22 }
```

#### LINE BY LINE

|       |                                                                                                                            |
|-------|----------------------------------------------------------------------------------------------------------------------------|
| 9     | Walks while there's a current link and a next one to compare. This also handles the empty-list case.                       |
| 11    | <code>!(*cmp)(...)</code> means "not in the right order". The <code>!</code> is easy to forget and flips the whole result. |
| 13–15 | Swaps the <b>data</b> of the two links. The links stay where they are.                                                     |
| 16    | Back to the start, because the swap might have disturbed something further back.                                           |
| 18–19 | If they were fine, move to the next link.                                                                                  |
| 21    | Returns the same head it received: since only data was swapped, it never changes.                                          |

## L4 flood\_fill

Function that fills a connected region of a 2D map.

#### WHAT THE SUBJECT ASKS FOR

Write a function that receives a `char **` (a two-dimensional map), a `t_point` with its dimensions, and a `t_point` with the starting point.

Starting from that point, it fills the whole region by replacing the characters with `'F'`. A region is a group of the same character bounded horizontally and vertically by other characters or by the edge of the map. **It does not fill diagonally.**

The declaration is `void flood_fill(char **tab, t_point size, t_point begin);`

## HOW TO THINK ABOUT IT

- 1 The problem solves itself with recursion: fill the current cell and ask the same algorithm to fill the four neighbours.
- 2 A recursive function always needs stopping cases, and here there are two: stepping outside the map, and landing on a cell that isn't the colour being filled.
- 3 Filling the cell **before** calling its neighbours is what keeps the program from going in circles: when a neighbour tries to come back, it no longer finds the original colour there.
- 4 A note on indices: `tab[y][x]` — the row is `y` and the column is `x`, in that order.

## THE CODE

### flood\_fill.c

```
1  #include "flood_fill.h"
2
3  void  fill(char **tab, t_point size, t_point p, char target)
4  {
5      t_point next;
6
7      if (p.x < 0 || p.x >= size.x || p.y < 0 || p.y >= size.y)
8          return ;
9      if (tab[p.y][p.x] != target)
10         return ;
11     tab[p.y][p.x] = 'F';
12     next.y = p.y;
13     next.x = p.x + 1;
14     fill(tab, size, next, target);
15     next.x = p.x - 1;
16     fill(tab, size, next, target);
17     next.x = p.x;
18     next.y = p.y + 1;
19     fill(tab, size, next, target);
20     next.y = p.y - 1;
21     fill(tab, size, next, target);
22 }
23
24 void  flood_fill(char **tab, t_point size, t_point begin)
25 {
26     if (tab[begin.y][begin.x] == 'F')
27         return ;
28     fill(tab, size, begin, tab[begin.y][begin.x]);
29 }
```

### flood\_fill.h

```
1  #ifndef FLOOD_FILL_H
2  # define FLOOD_FILL_H
3
4  typedef struct s_point
5  {
6      int x;
7      int y;
8  } t_point;
9
10 void  fill(char **tab, t_point size, t_point p, char target);
11 void  flood_fill(char **tab, t_point size, t_point begin);
12
13 #endif
```

## LINE BY LINE

|       |                                                                                                                                           |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 7-8   | First stopping case: outside the map.                                                                                                     |
| 9-10  | Second stopping case: this cell isn't the colour being filled.                                                                            |
| 11    | Fills it. This has to happen before the following calls, or the program loops forever.                                                    |
| 12-21 | The four neighbours: right, left, down, up. The <code>next</code> variable is reused each time.                                           |
| 26-27 | If the starting point is already <code>'F'</code> , there's nothing to do — and this guard also prevents infinite recursion in that case. |
| 28    | Kicks off the recursion, passing as the target the colour that's at the starting point.                                                   |

## L4 check\_mate

Say whether the king is in check on a given board.

### WHAT THE SUBJECT ASKS FOR

The program receives the rows of a chessboard as arguments and checks whether the king is in check. Only pawns (`P`), bishops (`B`), rooks (`R`), one queen (`Q`) and one king (`K`) exist. Everything else counts as an empty square.

The board is always square, but its size varies. There's only one king and every other piece is an enemy. A piece only captures the **first** piece it finds along its path.

If the king can be captured, show `Success` followed by a newline; otherwise `Fail`. With no arguments, show only a newline.

Allowed functions: `write`, `malloc`, `free`.

```
$> ./check_mate 'R...' '..P.' '.K..' '....' | cat -e
Success$
$> ./check_mate '..' '.K' | cat -e
Fail$
```

### HOW TO THINK ABOUT IT

- 1 Instead of asking "which pieces attack the king?", ask the question backwards: **starting from the king, what's in each direction?** That way you don't need to scan the whole board for every piece.
- 2 There are eight directions: four straight (where rooks and the queen attack) and four diagonal (bishops and the queen). In each one, walk until you find a piece or fall off the board. **The first piece found blocks the direction**, whether it attacks or not.
- 3 The pawn is the one special case, because it only captures one square away and diagonally. In this subject, the pawn captures downward — you can see it from the second example, where the `P` sits on the row above the king.
- 4 The board doesn't need to be copied: `argv + 1` is already a `char **` with all the rows.

## THE CODE

`check_mate.c`

```

1  #include <unistd.h>
2  #include "check_mate.h"
3
4  int    is_piece(char c)
5  {
6      return (c == 'P' || c == 'B' || c == 'R' || c == 'Q');
7  }
8
9  int    ray(char **b, int n, t_pos k, t_pos d)
10 {
11     int r;
12     int c;
13
14     r = k.r + d.r;
15     c = k.c + d.c;
16     while (r >= 0 && r < n && c >= 0 && c < n)
17     {
18         if (is_piece(b[r][c]))
19         {
20             if (d.r == 0 || d.c == 0)
21                 return (b[r][c] == 'R' || b[r][c] == 'Q');
22             return (b[r][c] == 'B' || b[r][c] == 'Q');
23         }
24         r = r + d.r;
25         c = c + d.c;
26     }
27     return (0);
28 }
29
30 int    in_check(char **b, int n, t_pos k)
31 {
32     int    dr[8] = {-1, 1, 0, 0, -1, -1, 1, 1};
33     int    dc[8] = {0, 0, -1, 1, -1, 1, -1, 1};
34     t_pos  d;
35     int    i;
36
37     i = 0;
38     while (i < 8)
39     {
40         d.r = dr[i];
41         d.c = dc[i];
42         if (ray(b, n, k, d))
43             return (1);
44         i++;
45     }
46     if (k.r > 0 && k.c > 0 && b[k.r - 1][k.c - 1] == 'P')
47         return (1);
48     if (k.r > 0 && k.c + 1 < n && b[k.r - 1][k.c + 1] == 'P')
49         return (1);
50     return (0);
51 }
52
53 int    main(int argc, char **argv)
54 {
55     char    **b;
56     int     n;
57     t_pos   k;
58
59     if (argc < 2)
60     {
61         write(1, "\n", 1);
62         return (0);
63     }
64     b = argv + 1;
65     n = argc - 1;
66     k.r = 0;
67     while (k.r < n)

```

```

68     {
69         k.c = 0;
70         while (b[k.r][k.c])
71         {
72             if (b[k.r][k.c] == 'K' && in_check(b, n, k))
73             {
74                 write(1, "Success\n", 8);
75                 return (0);
76             }
77             k.c++;
78         }
79         k.r++;
80     }
81     write(1, "Fail\n", 5);
82     return (0);
83 }

```

#### check\_mate.h

```

1  #ifndef CHECK_MATE_H
2  # define CHECK_MATE_H
3
4  typedef struct s_pos
5  {
6      int r;
7      int c;
8  } t_pos;
9
10 int is_piece(char c);
11 int ray(char **b, int n, t_pos k, t_pos d);
12 int in_check(char **b, int n, t_pos k);
13
14 #endif

```

#### LINE BY LINE

- 4-7 What counts as a piece. Everything else is an empty square, as the subject says.
- 9 One direction at a time. `d` says how much to move in row and column at each step.
- 14-15 Starts at the square next to the king, not the king itself.
- 16 Walks while still inside the board.
- 18 Found a piece: whatever it is, this direction ends here.
- 20-21 If the direction is straight (one of its components is zero), only the rook and the queen attack.
- 22 Otherwise it's diagonal, and only the bishop and the queen attack.
- 32-33 The eight directions, stored in two arrays: the first four straight, the last four diagonal.
- 38-45 Tests all eight. One attacking is enough for the king to be in check.
- 46-49 The two possible pawns: on the row above, one column to each side. The `k.r > 0` and `k.c + 1 < n` tests keep you from reading outside the board.
- 64-65 The board is the arguments from the first one onward, and its size is how many there are.
- 67-79 Looks for the king. As soon as it's found, it's tested.

The pawn's capture direction is the only ambiguous point in this subject. Here the second example from the subject itself was followed, which only gives `Success` if the pawn captures downward. If your subject comes with an `examples.txt` showing a different diagram, the lines to swap are 46 and 48.

## L5 Level 5

6 exercises. The hardest in the pool. If one of these comes up in the exam, you already have a passing grade.

### L5 brackets

Check whether each argument's brackets are properly closed.

#### WHAT THE SUBJECT ASKS FOR

For each argument, show `OK` followed by a newline if the expression is properly paired, otherwise `Error` followed by a newline.

`(` and `)`, `[` and `]`, and `{` and `}` count as brackets. Every other character is ignored. An open bracket has to be closed by the matching one, in the right order. A string with no brackets at all is properly paired.

With no arguments, show only a newline.

Allowed functions: `write`.

```
$> ./brackets '(johndoe)' | cat -e
OK$
$> ./brackets '([)]' | cat -e
Error$
```

#### HOW TO THINK ABOUT IT

- 1 The right structure for this is called a **stack**: a list where you only add and remove from the top, like a stack of plates.
- 2 The idea: when an open bracket shows up, store on top which **closing** bracket is expected. When a closing one shows up, compare it with the top and remove it.
- 3 If what shows up doesn't match the top, it's wrong. If a closing bracket shows up with an empty stack, also wrong. If anything is left on the stack at the end, also wrong.
- 4 The third case is what tells `([)]` apart from `([])`: they have the same characters, only the order changes, and it's the top of the stack that catches it.
- 5 A stack is built with an array and an index for the top. No `malloc` needed.

## THE CODE

brackets.c

```
1  #include <unistd.h>
2  #include "brackets.h"
3
4  int    is_open(char c)
5  {
6      return (c == '(' || c == '[' || c == '{');
7  }
8
9  char    closer(char c)
10 {
11     if (c == '(')
12         return (')');
13     if (c == '[')
14         return (']');
15     return ('}');
16 }
17
18 int    check(char *s)
19 {
20     char    stack[4096];
21     int      top;
22     int      i;
23
24     top = 0;
25     i = 0;
26     while (s[i])
27     {
28         if (is_open(s[i]) && top < 4096)
29             stack[top++] = closer(s[i]);
30         else if (s[i] == ')' || s[i] == ']' || s[i] == '}')
31         {
32             if (top == 0 || stack[--top] != s[i])
33                 return (0);
34         }
35         i++;
36     }
37     return (top == 0);
38 }
39
40 int    main(int argc, char **argv)
41 {
42     int i;
43
44     if (argc < 2)
45     {
46         write(1, "\n", 1);
47         return (0);
48     }
49     i = 1;
50     while (i < argc)
51     {
52         if (check(argv[i]))
53             write(1, "OK\n", 3);
54         else
55             write(1, "Error\n", 6);
56         i++;
57     }
58     return (0);
59 }
```

```

1  #ifndef BRACKETS_H
2  # define BRACKETS_H
3
4  int      is_open(char c);
5  char     closer(char c);
6  int      check(char *s);
7
8  #endif

```

## LINE BY LINE

- 4-7 The three characters that open.
- 9-16 Given one that opens, which one closes it. The final `return` covers `{` because there are no more options left.
- 20-21 The stack and the top index. `top` is always "how many elements are in there".
- 28-29 Opened: stores the expected closing bracket on top. `stack[top++]` stores it and **then** raises the top.
- 30 Closed: only does something if it's really one of the three closing characters.
- 32 Two ways to fail in one line: the stack is empty, or the top isn't the expected closer. `stack[--top]` lowers the top and **then** reads — that's the pop operation.
- 37 At the end it has to be empty. If something's still open, it's an error.
- 50-56 One `OK` or `Error` per argument.

## L5 ft\_itoa\_base

Function that converts an integer into a string, in any base from 2 to 16.

## WHAT THE SUBJECT ASKS FOR

Write a function that converts an integer value into a string terminated by zero, in the given base, storing the result in memory the function has to allocate.

The base is an integer from 2 to 16. The characters are the digits 0 through 9 followed by **uppercase** A through F.

If the base is 10 and the value is negative, the string carries a `-` at the front. **In any other base the value is treated as unsigned.**

The declaration is `char *ft_itoa_base(int value, int base);`

Allowed functions: `malloc`.

## HOW TO THINK ABOUT IT

- 1 It's `ft_itoa` from level 4 with the base as a variable, plus the special sign rule.
- 2 The sign rule is what makes this harder: only in base 10 does a negative number carry a `-`. In the other bases, the same number has to be read as a huge positive value — that's why you work with `unsigned int` instead of `int`.
- 3 `-(unsigned int)value` is the safe way to get the absolute value. Writing `-value` would blow up on the smallest possible `int`, which has no counterpart.
- 4 The rest is the usual pattern: count digits, request memory, fill in back to front.

## THE CODE

ft\_itoa\_base.c

```
1  #include <stdlib.h>
2
3  char  *ft_itoa_base(int value, int base)
4  {
5      char      *s;
6      char      *digits;
7      unsigned int  n;
8      int        len;
9      int        neg;
10
11     digits = "0123456789ABCDEF";
12     neg = (base == 10 && value < 0);
13     n = (unsigned int)value;
14     if (neg)
15         n = -(unsigned int)value;
16     len = 1;
17     while (n >= (unsigned int)base)
18     {
19         n = n / (unsigned int)base;
20         len++;
21     }
22     len = len + neg;
23     s = malloc(sizeof(char) * (len + 1));
24     if (!s)
25         return (0);
26     s[len] = '\0';
27     n = (unsigned int)value;
28     if (neg)
29         n = -(unsigned int)value;
30     while (len-- > neg)
31     {
32         s[len] = digits[n % (unsigned int)base];
33         n = n / (unsigned int)base;
34     }
35     if (neg)
36         s[0] = '-';
37     return (s);
38 }
```

### LINE BY LINE

- |       |                                                                                                                                                                      |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 11    | The digit table, with uppercase as the subject asks.                                                                                                                 |
| 12    | The sign only exists in base 10. This line is the subject's rule, written as-is.                                                                                     |
| 13-15 | The number becomes an <code>unsigned int</code> . In negative base 10 the absolute value is used; in the others, the value is used as-is, reinterpreted as positive. |
| 17-21 | Counts how many digits it has in this base. Same loop as <code>ft_itoa</code> with <code>base</code> instead of 10.                                                  |
| 22    | One more character if there's a sign.                                                                                                                                |
| 23    | Requests the space, with the <code>+ 1</code> for the <code>'\0'</code> .                                                                                            |
| 27-29 | Restores the value, because the counting loop destroyed it.                                                                                                          |
| 30    | <code>len-- &gt; neg</code> compares and then steps back; it stops at <code>neg</code> , leaving the sign's position free.                                           |
| 32    | The remainder of dividing by the base gives a number between 0 and <code>base - 1</code> , which is exactly the right position in the table.                         |

## L5 print\_memory

Function that shows a block of memory in hexadecimal and as text.

### WHAT THE SUBJECT ASKS FOR

Write a function that receives an address and a size and shows the memory as in the subject's example: each line has 16 bytes in hexadecimal, grouped two by two, followed by those same 16 bytes as text.

Bytes that aren't printable characters show as `.`. The last line, if it has fewer than 16 bytes, gets spaces in place of the missing ones so the text column stays aligned.

The declaration is `void print_memory(const void *addr, size_t size);`

Allowed functions: `write`.

```
0000 0000 1700 0000 9600 0000 ff00 0000 .....  
0c00 0000 1000 0000 1500 0000 2a00 0000 .....*..  
0000 0000 0000 0000 .....  
.....
```

### HOW TO THINK ABOUT IT

- 1 There's no algorithm to this exercise: it's all formatting. You win or lose on the alignment.
- 2 Three nested loops: one per line of 16 bytes, one for the hexadecimal part, and one for the text part.
- 3 `const void *` is an address with no type. To read it byte by byte, it's copied into an `unsigned char *` — that way each position is exactly one byte.
- 4 The detail that makes the last line line up: once there are no more bytes, **two spaces** are written in place of the two hex digits. That keeps the text column starting in the same spot every time.

## THE CODE

print\_memory.c

```
1  #include <unistd.h>
2
3  void    ft_putchar(char c)
4  {
5      write(1, &c, 1);
6  }
7
8  void    put_hex_byte(unsigned char b)
9  {
10     char    *digits;
11
12     digits = "0123456789abcdef";
13     ft_putchar(digits[b / 16]);
14     ft_putchar(digits[b % 16]);
15 }
16
17 void    print_memory(const void *addr, size_t size)
18 {
19     const unsigned char *p;
20     size_t                i;
21     size_t                j;
22
23     p = addr;
24     i = 0;
25     while (i < size)
26     {
27         j = 0;
28         while (j < 16)
29         {
30             if (i + j < size)
31                 put_hex_byte(p[i + j]);
32             else
33                 write(1, " ", 2);
34             if (j % 2 == 1)
35                 ft_putchar(' ');
36             j++;
37         }
38         j = 0;
39         while (j < 16 && i + j < size)
40         {
41             if (p[i + j] >= 32 && p[i + j] <= 126)
42                 ft_putchar(p[i + j]);
43             else
44                 ft_putchar('.');
45             j++;
46         }
47         ft_putchar('\n');
48         i = i + 16;
49     }
50 }
```

## LINE BY LINE

8–15 Writes a byte as two hex digits. `b / 16` is the left digit and `b % 16` the right one.

19 The typed pointer, so it can be read byte by byte.

25 One loop per line of 16 bytes.

28 The hex-part loop: always runs 16 times, whether there are bytes left or not.

30–31 If there's still a byte, write it.

32–33 Otherwise write two spaces, to keep the alignment.

34–35 A space every two bytes, which is the subject's grouping.

39 The text loop: this one stops once the bytes run out, it doesn't pad anything.

41–42 Printable characters range from 32 to 126.

44 Everything else comes out as a dot.

48 Skips 16 bytes for the next line.

## L5 options

Combine command-line options into an integer and show it in binary.

### WHAT THE SUBJECT ASKS FOR

The program receives an unspecified number of arguments that are options, and writes a representation of those options in byte groups, followed by a newline.

An option is an argument that starts with `-` and has one or more characters from `a` to `z`. All the options are stored in a single `int`, where each option is a bit: `a` is bit 0, `b` is bit 1, and so on up to `z`.

With no arguments, or with the `-h` option active, a usage line is shown. An invalid option writes `Invalid Option` and a newline.

Allowed functions: `write`.

```
$> ./options -abc -ijk
00000000 00000000 00000111 00000111
$> ./options -z
00000010 00000000 00000000 00000000
$> ./options -%
Invalid Option
```

### HOW TO THINK ABOUT IT

- 1 The central idea: an `int` has 32 bits and the alphabet has 26 letters, so there's room for one letter per bit.
- 2 Turning on the bit for letter `c` is written `flags = flags | (1 << (c - 'a'))`. `c - 'a'` gives the letter's position (0 for `a`), `1 <<` puts a 1 in that position, and `|` switches it on without erasing the others.
- 3 Writing it in binary is `print_bits` from level 2 stretched to 32 bits, with a space every eight.
- 4 `-h` takes priority: as soon as it shows up, the usage is shown and the program ends, even if other options were already read.

## THE CODE

`options.c`

```

1  #include <unistd.h>
2  #include "options.h"
3
4  void    ft_putstr(char *s)
5  {
6      int i;
7
8      i = 0;
9      while (s[i])
10         i++;
11     write(1, s, i);
12 }
13
14 void    put_flags(unsigned int n)
15 {
16     int    i;
17     char    c;
18
19     i = 31;
20     while (i >= 0)
21     {
22         c = ((n >> i) & 1) + '0';
23         write(1, &c, 1);
24         if (i % 8 == 0 && i != 0)
25             write(1, " ", 1);
26         i--;
27     }
28     write(1, "\n", 1);
29 }
30
31 void    usage(void)
32 {
33     ft_putstr("options: abcdefghijklmnopqrstuvwxyz\n");
34 }
35
36 int     main(int argc, char **argv)
37 {
38     unsigned int    flags;
39     int             i;
40     int             j;
41
42     flags = 0;
43     if (argc == 1)
44     {
45         usage();
46         return (0);
47     }
48     i = 1;
49     while (i < argc)
50     {
51         j = 1;
52         if (argv[i][0] != '-' || argv[i][1] == '\0')
53         {
54             ft_putstr("Invalid Option\n");
55             return (0);
56         }
57         while (argv[i][j])
58         {
59             if (argv[i][j] < 'a' || argv[i][j] > 'z')
60             {
61                 ft_putstr("Invalid Option\n");
62                 return (0);
63             }
64             if (argv[i][j] == 'h')
65             {
66                 usage();
67                 return (0);

```

```
68     }
69     flags = flags | (1u << (argv[i][j] - 'a'));
70     j++;
71 }
72 i++;
73 }
74 put_flags(flags);
75 return (0);
76 }
```

options.h

```
1 #ifndef OPTIONS_H
2 # define OPTIONS_H
3
4 void ft_putstr(char *s);
5 void put_flags(unsigned int n);
6 void usage(void);
7
8 #endif
```

## LINE BY LINE

|       |                                                                                                                      |
|-------|----------------------------------------------------------------------------------------------------------------------|
| 14-29 | The 32-bit version of <code>print_bits</code> . Starts at bit 31 and counts down.                                    |
| 22    | The same recipe as always to read bit <code>i</code> , plus <code>+ '0'</code> to turn it into a character.          |
| 24    | The space between groups of eight. <code>i != 0</code> avoids an extra space at the end.                             |
| 43-47 | With no arguments, the usage is shown.                                                                               |
| 52    | An argument that doesn't start with <code>-</code> , or that's just <code>-</code> , is invalid.                     |
| 59-63 | Any character outside <code>a</code> to <code>z</code> is invalid.                                                   |
| 64-68 | <code>-h</code> interrupts everything and shows the usage.                                                           |
| 69    | Turns on the letter's bit. <code>1u</code> is the number 1 with no sign — the correct choice when working with bits. |

**L5** **brainfuck** Interpreter for the Brainfuck language.

Interpreter for the Brainfuck language.

### WHAT THE SUBJECT ASKS FOR

Write a Brainfuck interpreter. The source code comes in the first argument and is always valid.

The language has a 2048-byte tape starting at zero and a pointer to the first byte. The operators are: `>` moves the pointer forward; `<` moves it back; `+` increments the pointed byte; `-` decrements it; `.` writes the pointed byte; `[` jumps to the matching `]` if the pointed byte is zero; `]`  jumps back to the matching `[` if the pointed byte isn't zero. Any other character is a comment.

Allowed functions: `write`, `malloc`, `free`.

```
$> ./brainfuck "+++++++[>+++++>+++++++>++++<<<<-]>+.,+.,+++++.,++.,+.,<<+++++++>.,++.,-
-----.,----->+.,." | cat -e
Hello World!$
```

## HOW TO THINK ABOUT IT

- 1 Six of the eight operators are one line each: a chain of `else if` handles them.
- 2 The difficulty is only in the square brackets, because you need to find the **matching** one, and there can be loops inside loops.
- 3 The solution is to count depth: searching forward, start at 1, each new `[` adds one and each `]` subtracts one; when it reaches zero, that's the match. Searching backward does the same thing in reverse.

- ④ Two variables not to confuse: `pc` is where you are in the **code**, `p` is where you are on the **tape**.

## THE CODE

`brainfuck.c`

```

1  #include <unistd.h>
2  #include <stdlib.h>
3  #include "brainfuck.h"
4
5  void  jump_fwd(char *code, int *pc)
6  {
7      int depth;
8
9      depth = 1;
10     while (depth > 0 && code[*pc])
11     {
12         (*pc)++;
13         if (code[*pc] == '[')
14             depth++;
15         else if (code[*pc] == ']')
16             depth--;
17     }
18 }
19
20 void  jump_back(char *code, int *pc)
21 {
22     int depth;
23
24     depth = 1;
25     while (depth > 0 && *pc > 0)
26     {
27         (*pc)--;
28         if (code[*pc] == ']')
29             depth++;
30         else if (code[*pc] == '[')
31             depth--;
32     }
33 }
34
35 int  main(int argc, char **argv)
36 {
37     char    *mem;
38     char    *code;
39     int     p;
40     int     pc;
41
42     if (argc != 2)
43     {
44         write(1, "\n", 1);
45         return (0);
46     }
47     mem = malloc(2048);
48     if (!mem)
49         return (1);
50     p = 0;
51     while (p < 2048)
52         mem[p++] = 0;
53     p = 0;
54     pc = 0;
55     code = argv[1];
56     while (code[pc])
57     {
58         if (code[pc] == '>')
59             p++;
60         else if (code[pc] == '<')
61             p--;
62         else if (code[pc] == '+')
63             mem[p]++;
64         else if (code[pc] == '-')
65             mem[p]--;
66         else if (code[pc] == '.')
67             write(1, &mem[p], 1);

```

```

68     else if (code[pc] == '[' && mem[p] == 0)
69         jump_fwd(code, &pc);
70     else if (code[pc] == ']' && mem[p] != 0)
71         jump_back(code, &pc);
72     pc++;
73 }
74 free(mem);
75 return (0);
76 }

```

#### brainfuck.h

```

1  #ifndef BRAINFUCK_H
2  # define BRAINFUCK_H
3
4  void  jump_fwd(char *code, int *pc);
5  void  jump_back(char *code, int *pc);
6
7  #endif

```

#### LINE BY LINE

- 5-18 Searches for the matching `]`, going forward.
- 9 Starts at 1 because you're already standing on a `[`.
- 13-16 Every `[` increases the depth, every `]` decreases it. When it reaches zero, it stopped in the right place.
- 20-33 The same thing in reverse, going backward.
- 47-52 The tape. `malloc` doesn't zero it out, so the loop on line 51 has to do it by hand.
- 58-65 The four movement and value operators, one per line.
- 66-67 The `.` writes the pointed byte. `&mem[p]` is its address.
- 68-69 The `[` only jumps if the byte is zero; otherwise it does nothing and the loop continues normally.
- 70-71 The `]` only jumps back if the byte isn't zero.
- 72 This `pc++` always runs, even after a jump. It's what makes the program keep going from the matching bracket.

Any character that isn't one of the eight operators doesn't match any `else if` and is simply ignored — comments are handled without a single line of code for them.

## L5 rpn\_calc

Evaluate an expression in reverse Polish notation.

#### WHAT THE SUBJECT ASKS FOR

Receive an expression written in reverse Polish notation in the first argument, evaluate it, and show the result followed by a newline.

In reverse Polish notation the operator comes **after** its two operands: `3 4 +` is 7. Each operator consumes the two previous values and the result takes their place. Operands and operators are separated by at least one space.

You have to implement `+`, `-`, `*`, `/` and `%`. If the expression isn't valid, or if there isn't exactly one argument, show `Error` followed by a newline.

Allowed functions: `atoi`, `printf`, `write`, `malloc`, `free`.

```
$> ./rpn_calc "1 2 * 3 * 4 +" | cat -e
10$
$> ./rpn_calc "1 2 3 4 +" | cat -e
Error$
```

#### HOW TO THINK ABOUT IT

- ❶ Another stack, like in `brackets`, but of numbers this time.
- ❷ The rule is simple: if the piece you read is a number, push it; if it's an operator, pop the top two, do the calculation, and push the result.
- ❸ At the end there has to be **exactly one** value left. That's what makes the second example an error: four numbers are left and no operator to consume them.
- ❹ To split the string into pieces without library functions: skip spaces, measure up to the next space, and advance by that much.
- ❺ Deciding whether a piece is a number or an operator before using it is what catches every invalid input with a single check.

## THE CODE

rpn\_calc.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "rpn_calc.h"
4
5  int tok_len(char *s, int i)
6  {
7      int n;
8
9      n = 0;
10     while (s[i + n] && s[i + n] != ' ')
11         n++;
12     return (n);
13 }
14
15 int is_number(char *s, int i)
16 {
17     int n;
18     int k;
19
20     n = tok_len(s, i);
21     if (n == 0)
22         return (0);
23     k = 0;
24     if (s[i] == '-' && n > 1)
25         k = 1;
26     while (k < n)
27     {
28         if (s[i + k] < '0' || s[i + k] > '9')
29             return (0);
30         k++;
31     }
32     return (1);
33 }
34
35 int is_operator(char *s, int i)
36 {
37     if (tok_len(s, i) != 1)
38         return (0);
39     return (s[i] == '+' || s[i] == '-' || s[i] == '*'
40         || s[i] == '/' || s[i] == '%');
41 }
42
43 int apply(int a, int b, char op, int *err)
44 {
45     if (op == '+')
46         return (a + b);
47     if (op == '-')
48         return (a - b);
49     if (op == '*')
50         return (a * b);
51     if (b == 0)
52     {
53         *err = 1;
54         return (0);
55     }
56     if (op == '/')
57         return (a / b);
58     return (a % b);
59 }
60
61 int main(int argc, char **argv)
62 {
63     int stack[512];
64     int top;
65     int i;
66     int err;
67

```

```

68     err = 0;
69     top = 0;
70     i = 0;
71     if (argc != 2)
72         err = 1;
73     while (!err && argv[1][i])
74     {
75         while (argv[1][i] == ' ')
76             i++;
77         if (!argv[1][i])
78             break ;
79         if (is_number(argv[1], i) && top < 512)
80             stack[top++] = atoi(&argv[1][i]);
81         else if (is_operator(argv[1], i) && top >= 2)
82         {
83             stack[top - 2] = apply(stack[top - 2], stack[top - 1],
84                                   argv[1][i], &err);
85             top--;
86         }
87         else
88             err = 1;
89         i = i + tok_len(argv[1], i);
90     }
91     if (err || top != 1)
92         printf("Error\n");
93     else
94         printf("%d\n", stack[0]);
95     return (0);
96 }

```

#### rpn\_calc.h

```

1  #ifndef RPN_CALC_H
2  # define RPN_CALC_H
3
4  int  tok_len(char *s, int i);
5  int  is_number(char *s, int i);
6  int  is_operator(char *s, int i);
7  int  apply(int a, int b, char op, int *err);
8
9  #endif

```

#### LINE BY LINE

- |       |                                                                                                                                         |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 5-13  | Measures the piece starting at position <code>i</code> : counts up to the next space or the end.                                        |
| 15-33 | Is it a number? Accepts a leading <code>-</code> , but only if digits follow — that way a lone <code>-</code> doesn't pass as a number. |
| 35-41 | Is it an operator? It has to be exactly one character and one of the five.                                                              |
| 43-59 | Does the calculation. Division and remainder by zero flag an error instead of crashing the program.                                     |
| 63-64 | The number stack and the top index.                                                                                                     |
| 73    | The loop stops as soon as there's an error, without reading the rest.                                                                   |
| 75-78 | Skips the spaces. If it reaches the end, it's done.                                                                                     |
| 79-80 | Number: push it. <code>atoi</code> reads from the current position and stops by itself at the space.                                    |
| 81-86 | Operator: applies it to the top two. The result takes the position of the first of the two, and the top drops by one.                   |
| 87-88 | Neither a number nor an operator, or there weren't two values for the operator: error.                                                  |
| 89    | Advances exactly the length of the piece that was read.                                                                                 |
| 91-92 | Error if something went wrong, or if exactly one value wasn't left over.                                                                |

The subject doesn't say what to do on a division by zero. Here `Error` was chosen, which is the safest option: any other choice would crash the program, and a program that crashes always fails.

# The cheat sheet

One page with everything worth knowing by heart. Print just this one.

## THE STARTING POINT — WRITE THIS FIRST, ALWAYS

```
#include <unistd.h>

void ft_putchar(char c)
{
    write(1, &c, 1);
}

int main(int argc, char **argv)
{
    int i;

    if (argc == 2)
    {
        i = 0;
        while (argv[1][i])
        {
            /* the exercise goes here */
            i++;
        }
        ft_putchar('\n');
        return (0);
    }
}
```

## WRITING A NUMBER

```
void ft_putnbr(int n)
{
    if (n < 0)
    {
        ft_putchar('-');
        n = -n;
    }
    if (n >= 10)
        ft_putnbr(n / 10);
    ft_putchar(n % 10 + '0');
}
```

## WORD SEPARATOR

```
int is_space(char c)
{
    return (c == ' ' || c == '\t');
}
```

## SKIP SEPARATORS, THEN THE WORD

```
while (is_space(s[i]))
    i++;
while (s[i] && !is_space(s[i]))
{
    ft_putchar(s[i]);
    i++;
}
```

## TEXT → NUMBER

```
n = 0;
i = 0;
while (s[i] >= '0' && s[i] <= '9')
{
    n = n * 10 + (s[i] - '0');
    i++;
}
```

## LETTER ARITHMETIC

|                          |                                             |
|--------------------------|---------------------------------------------|
| is lowercase?            | <code>c&gt;='a' &amp;&amp; c&lt;='z'</code> |
| is uppercase?            | <code>c&gt;='A' &amp;&amp; c&lt;='Z'</code> |
| is a digit?              | <code>c&gt;='0' &amp;&amp; c&lt;='9'</code> |
| lowercase → UPPERCASE    | <code>c - 32</code>                         |
| UPPERCASE → lowercase    | <code>c + 32</code>                         |
| position in the alphabet | <code>c - 'a'</code>                        |
| letter number <i>i</i>   | <code>'a' + i</code>                        |
| digit → number           | <code>c - '0'</code>                        |
| number → digit           | <code>n + '0'</code>                        |
| rotate <i>k</i> letters  | <code>'a' + (c - 'a' + k) % 26</code>       |
| mirror the letter        | <code>'z' - (c - 'a')</code>                |

## ARGC COUNTS THE PROGRAM NAME

|                     |                                     |
|---------------------|-------------------------------------|
| "1 argument"        | <code>argc == 2</code>              |
| "2 arguments"       | <code>argc == 3</code>              |
| "3 arguments"       | <code>argc == 4</code>              |
| first / last        | <code>argv[1] / argv[argc-1]</code> |
| received but unused | <code>(void)argv;</code>            |

## BITS

|                   |                                            |
|-------------------|--------------------------------------------|
| read bit <i>i</i> | <code>(o &gt;&gt; i) &amp; 1</code>        |
| swap halves       | <code>(o&gt;&gt;4)   (o&lt;&lt;4)</code>   |
| is a power of 2   | <code>n &amp;&amp; !(n &amp; (n-1))</code> |

## MALLOC — THREE STEPS, IN THIS ORDER

1. count    `len = ... (+1 if it's a string)`
2. request    `p = malloc(sizeof(t) * len);`  
              if (!p) return (0);
3. fill in    and set the '\0' by hand

## THE THREE MATH RECIPES

```
/* gcd of a and b */
while (b != 0)
{
    t = a % b;
    a = b;
    b = t;
}

/* lcm = a / gcd * b (divide first!) */

/* is it prime? */
i = 2;
while (i * i <= n)
{
    if (n % i == 0)
        return (0);
    i++;
}
```

**Before you submit:** zero warnings with `cc -Wall -Wextra -Werror` · every example through `| cat -e` · run with no arguments and with too many · delete the test `main` if the exercise is a function · in the folder, only the `Expected files`.

# Alphabetical index

All 70 exercises in alphabetical order, with the level of each. Hovering over a name here also shows the subject.

|   |                   |   |                  |   |                    |
|---|-------------------|---|------------------|---|--------------------|
| 3 | add_prime_sum     | 0 | ft_print_numbers | 3 | pgcd               |
| 0 | aff_a             | 1 | ft_putstr        | 2 | print_bits         |
| 0 | aff_first_param   | 3 | ft_range         | 3 | print_hex          |
| 0 | aff_last_param    | 3 | ft_rrange        | 5 | print_memory       |
| 0 | aff_z             | 4 | ft_split         | 1 | repeat_alpha       |
| 2 | alpha_mirror      | 2 | ft_strcmp        | 1 | rev_print          |
| 5 | brackets          | 1 | ft_strcpy        | 4 | rev_wstr           |
| 5 | brainfuck         | 2 | ft_strdup        | 2 | reverse_bits       |
| 4 | check_mate        | 1 | ft_strlen        | 4 | rostring           |
| 2 | do_op             | 2 | ft_strrev        | 1 | rot_13             |
| 3 | epur_str          | 1 | ft_swap          | 1 | rotone             |
| 3 | expand_str        | 0 | hello            | 5 | rpn_calc           |
| 1 | first_word        | 3 | hiddenp          | 3 | rstr_capitalizer   |
| 1 | fizzbuzz          | 2 | inter            | 1 | search_and_replace |
| 4 | flood_fill        | 2 | is_power_of_2    | 4 | sort_int_tab       |
| 4 | fprime            | 2 | last_word        | 4 | sort_list          |
| 2 | ft_atoi           | 3 | lcm              | 3 | str_capitalizer    |
| 3 | ft_atoi_base      | 0 | maff_alpha       | 2 | swap_bits          |
| 0 | ft_countdown      | 0 | maff_revalpha    | 3 | tab_mult           |
| 4 | ft_itoa           | 2 | max              | 1 | ulstr              |
| 5 | ft_itoa_base      | 0 | only_a           | 2 | union              |
| 4 | ft_list_foreach   | 0 | only_z           | 2 | wdmatch            |
| 4 | ft_list_remove_if | 5 | options          |   |                    |
| 3 | ft_list_size      | 3 | paramsum         |   |                    |

The 70 `.c` and `.h` files in this document compile with `gcc -Wall -Wextra -Werror` without a single warning, and pass 149 output comparisons taken from the subjects' own examples, plus the functions' return-value tests. The printed code is injected straight from those same tested files.

The pool may vary slightly between campuses and between years.